Data Integration

Query Planning with Global-as-View and Local-as-View

Felix Naumann

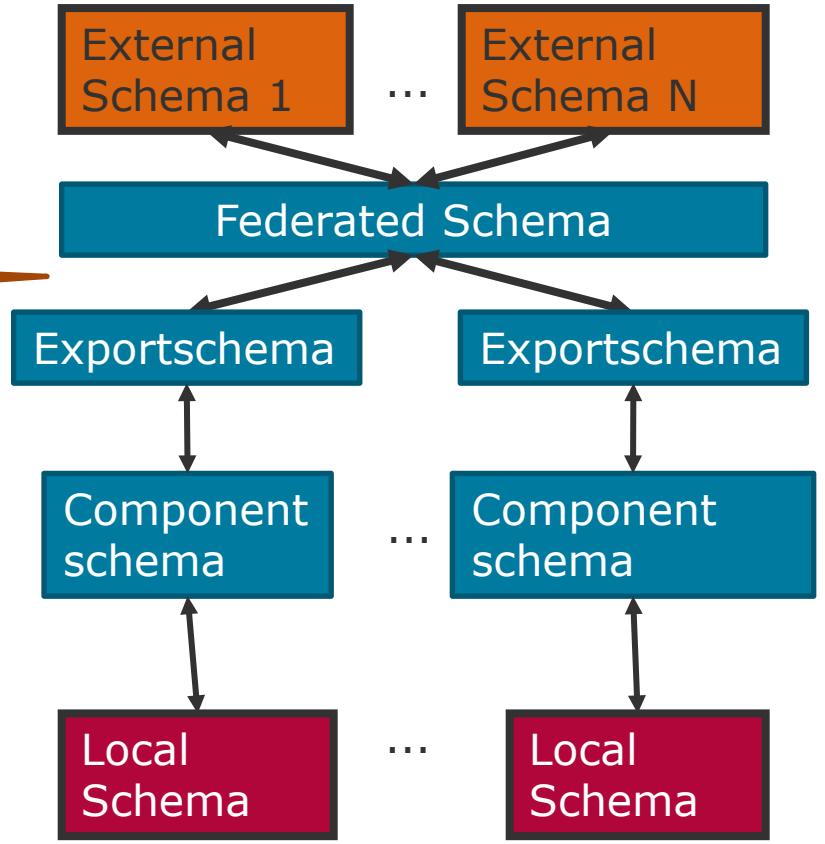
Overview

1. **Motivation**
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

Felix Naumann
Data Integration
Summer 2025

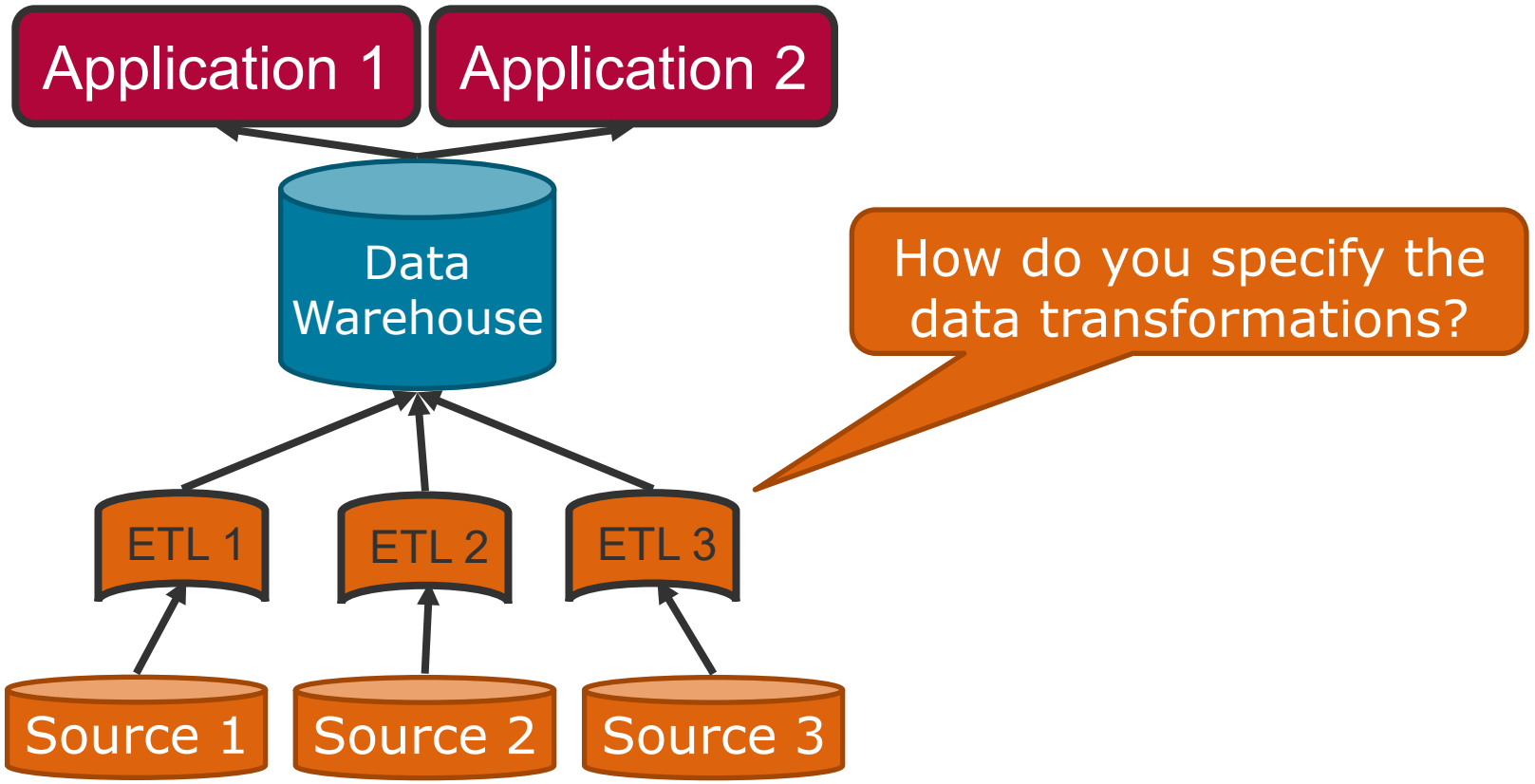
The Problem

How do you express this transition?



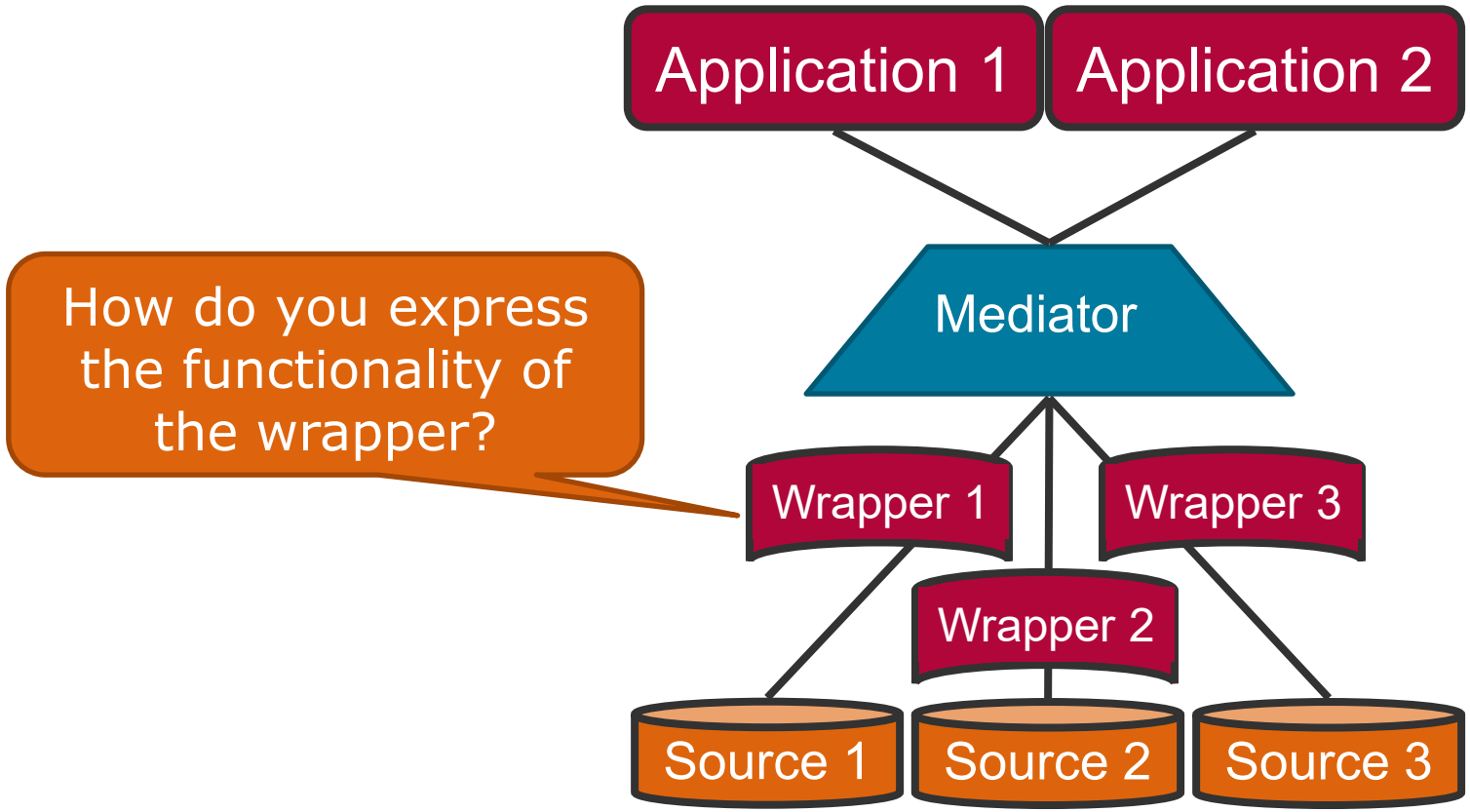
Felix Naumann
Data Integration
Summer 2025

The Problem



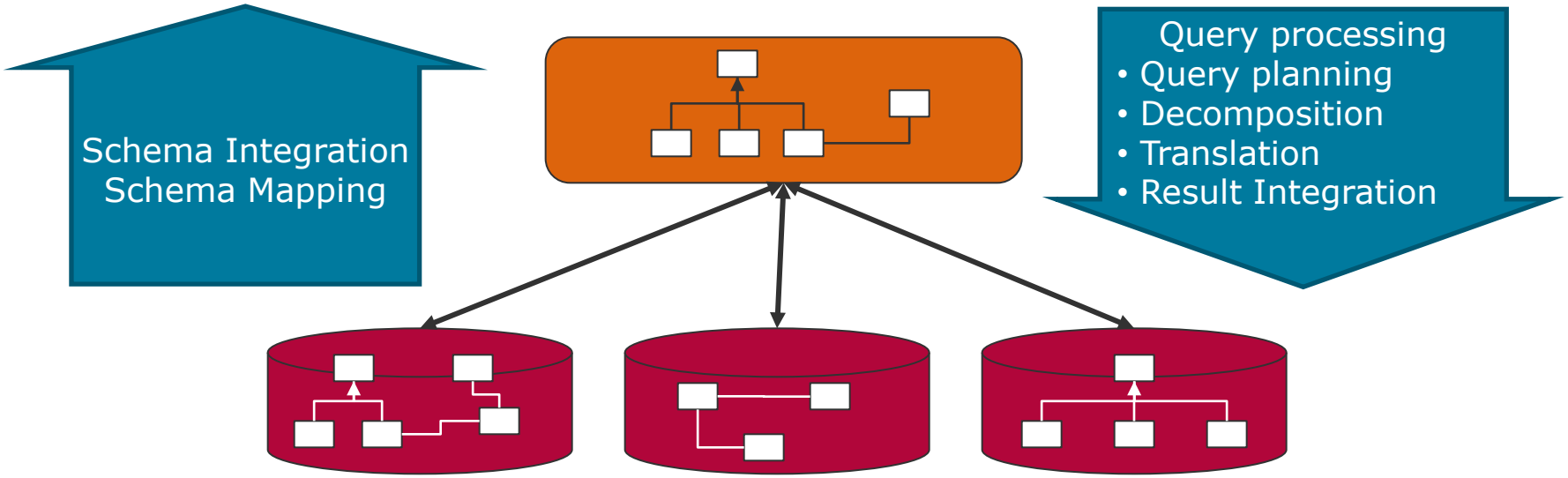
Felix Naumann
Data Integration
Summer 2025

The Problem



Felix Naumann
Data Integration
Summer 2025

More abstract



- If the integrated system provides a global schema
  - How do you describe the relationships between the global and the different local schemata?
  - How do you use these relationships for query planning?

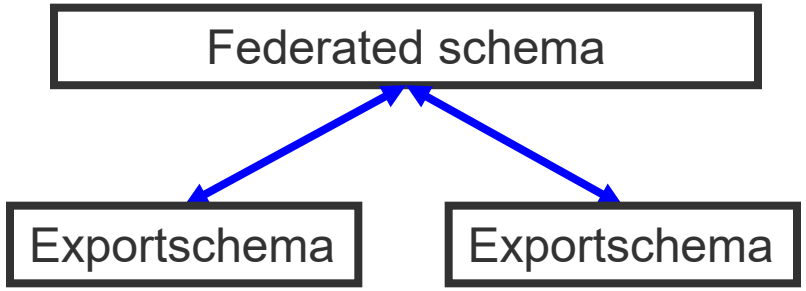
Overview

1. Motivation
2. **Correspondences**
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

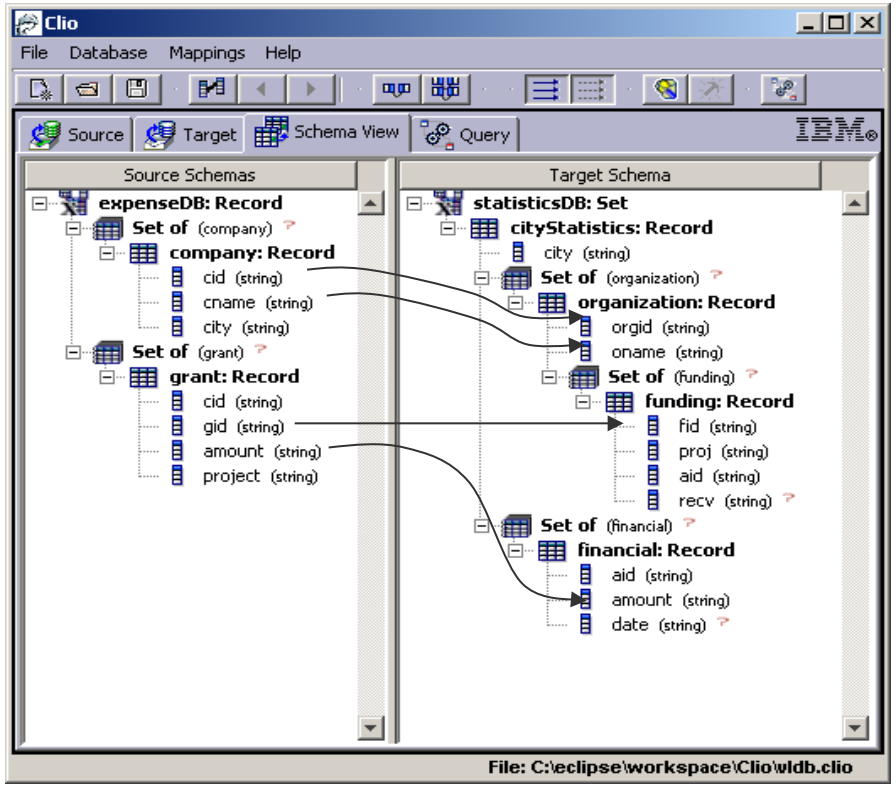
Felix Naumann
Data Integration
Summer 2025

Query-Correspondences

- Query-correspondence
  - Specification of semantically equivalent parts in source and target schema
- Rules with two parts
  1. Query to source
  2. Destinations in the target schema
  - In between: data transformation and restructuring
- Can use different "languages"
  - Global-as-View, Local-as-View, …
  - We prefer declarative correspondences
    - An XSLT program also does data transformation
    - But it is difficult to use for query planning
- Where do the correspondences come from?
  - Specify manually
  - (Semi-)automatic discovery: schema mapping/matching



Example




```
SELECT cid, cname          ⊆   SELECT orgid, oname
  FROM company                 FROM cityStatistics/organization

SELECT amount              ⊆   SELECT amount
  FROM grant                   FROM cityStatistics/financial
```

Felix Naumann
Data Integration
Summer 2025

Virtual and Materialized

- Translate queries (virtual)
  - Given a query to target schema
  - Find semantically equivalent set of queries to source schemata
  - Not applicable for materialized approaches

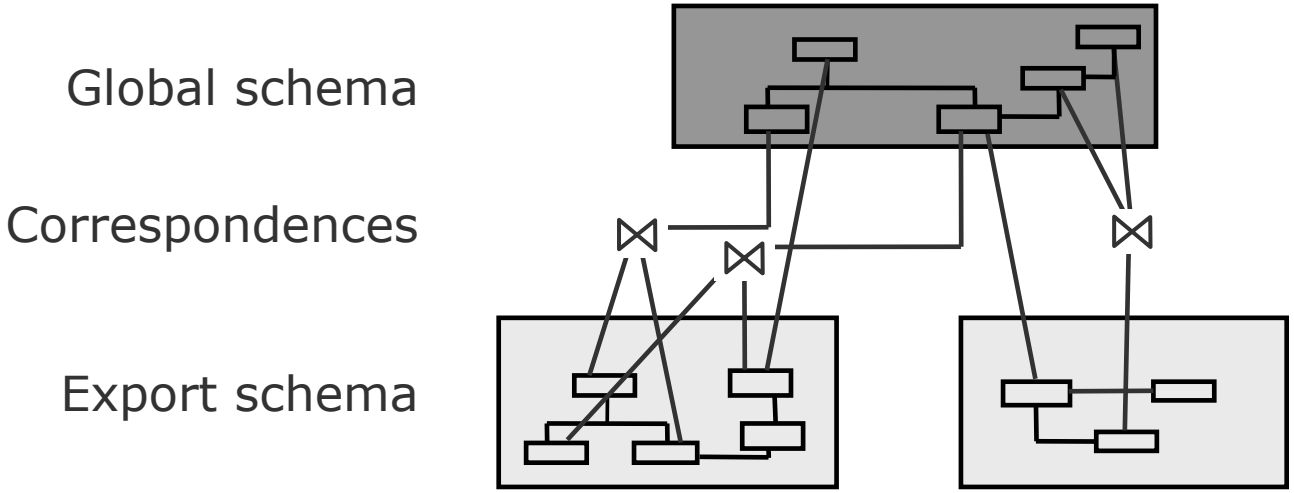
- Transform data from source to target
  - Given results of partial queries
    - or the entire database - migration
  - Transformation into target schema
  - Adaptation of values to the conventions of the target schema
  - Offline (ETL, DWH) or online (federation)

- Both variants should be able to use the same correspondences.

Overview

1. Motivation
2. Correspondences
3. **Overview: Query Planning**
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

Task of Query Processing



- Given a query *q* against the global schema
- Given a set of correspondences / logical mappings between global and local schemata
- Find all results for q.

Query processing – Overview

- Step 1: Query planning (in the next few weeks)
  - Which sources can / should / must be used?
  - Which parts of the query should be sent to which source?
- Step 2: Query translation
  - Translation from the language of the federation into the language of the sources
  - Wrapper
- Step 3: Query optimization
  - Find the appropriate execution order of partial queries
  - Who executes what (push)?
  - Who can execute what (compensation)?
- Step 4: Query execution
  - Monitoring, buffering, caching
  - Dynamic re-optimization
- Step 5: Result integration
  - Duplicate detection and conflict resolution

Step 1: Query planning

- Input
  - User query and query correspondences
- Task
  - Calculate query plans. A query plan is a set of logical queries to data sources, usually linked by join predicates.
  - Each plan must be executable and only calculate semantically correct results.
  - If there are several query plans, it is also the task of the query planning to decide which of them should be executed.
- Result
  - One or more query plans

Step 2: Query Translation

- Input
  - A query plan
- Task
  - A query plan consists of subqueries, each addressing exactly one data source. These subqueries are syntactically tailored to the integration layer.
  - Query translation must translate each subquery into a command that can be executed directly from the relevant data source.
- Result
  - A translated query plan

Step 3: Query Optimization

- Input
  - Translated query plan

- Task
  - Query plans consist of subqueries for which it is not yet clear in which order they are calculated and where query predicates contained in or connecting them are executed.
  - The determination of these points is the task of query optimization, the result of which is a (physical) execution plan.

- Result
  - An execution plan

Step 4: Query Execution

- Input
  - An execution plan
- Task
  - An execution plan can be executed directly, but its execution must be monitored because network connections are often unreliable.
  - Predicates of the user query that have not been delegated to a data source must be executed in the integration system.
- Result
  - A set of result tuples

Step 5: Result Integration

- Input
  - A set of result tuples per execution plan
- Task
  - The results of the individual execution plans are often redundant or inconsistent.
  - During result integration, the individual results are integrated into an overall result, which requires duplicate detection and resolving conflicts in the data.
- Result
  - The result of the user query
  - …finally!

Example - Movie Domain

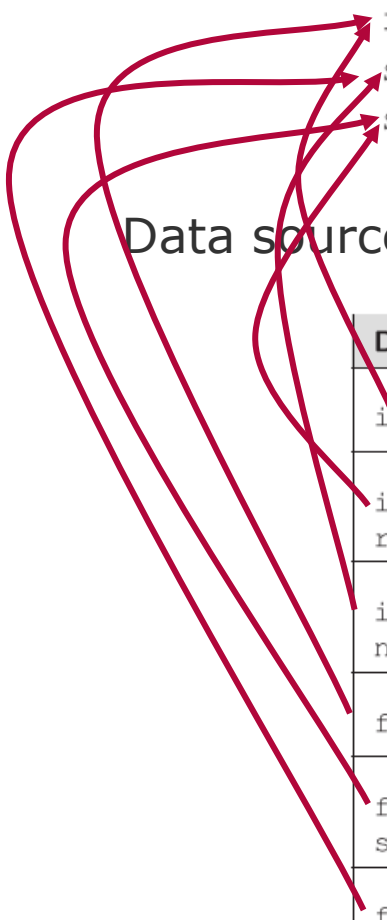
- Simplifying assumptions
  - Sources consist only of one (export) relation.
  - No data model heterogeneity
    - Assumption from now on
    - Task of the wrappers
  - For now: No structural or schematic heterogeneity
  - No semantic heterogeneity
    - Same names = same concepts
    - Only obvious spelling variants in the examples

Example

Global Schema

```
film( titel, regisseur)
schauspieler( schauspieler_name, nationalitaet)
spielt( titel, schauspieler_name, rolle)
```

Data sources



Datenquelle und exportierte Relation	Beschreibung	Globale Relation
`imdb.movie( title, director)`	Alle Filme der IMDB	`film`
`imdb.acts( title, actor_name, role)`	Zuordnung von Schauspielern zu Filmen der IMDB	`spielt`
`imdb.actor( actor_name, nationality)`	Alle Schauspieler der IMDB	`schauspieler`
`fd.film( titel, regisseur)`	Alle Filme des Filmdienstes	`film`
`fd.spielt( titel, schauspieler_name, rolle)`	Zuordnung von Schauspielern zu Filmen des Filmdienstes	`spielt`
`fd.schauspieler( schauspieler_name, nationalitaet)`	Alle Schauspieler des Filmdienstes	`schauspieler`

Query Planning

- Global query
  - „All films in which Hans Albers played a leading role “

```
SELECT titel, regisseur, rolle
FROM   film, spielt
WHERE  spielt.schauspieler_name = 'Hans Albers'
       AND spielt.rolle = 'Hauptrolle'
       AND spielt.titel = film.titel;
```

- Which sources are relevant?
  - **`film`**: sources **`imdb.movie`** and **`fd.film`**
  - **`spielt`**: sourcea **`imdb.acts`** and **`fd.spielt`**

- Yields four query plans (p_1 – p_4)

https://de.wikipedia.org/wiki/Hans_Albers#/media/Datei:Hans_Albers_1922.jpg

Plan	Anfrage
p_1	SELECT title, director, role FROM imdb.movie, imdb.acts WHERE imdb.acts.actor_name = 'Hans Albers' AND imdb.acts.role = 'main actor' AND imdb.acts.title = imdb.movie.title;
p_2	SELECT title, director, rolle FROM imdb.movie, fd.spielt WHERE fd.spielt.schauspieler_name = 'Hans Albers' AND fd.spielt.rolle = 'Hauptrolle' AND fd.spielt.titel = imdb.movie.title;
p_3	SELECT titel, regisseur, role FROM fd.film, imdb.acts WHERE imdb.acts.actor_name = 'Hans Albers' AND imdb.acts.role = 'main actor' AND fd.film.titel = imdb.acts.title;
p_4	SELECT titel, regisseur, rolle FROM fd.film, fd.spielt WHERE fd.spielt.schauspieler_name = 'Hans Albers' AND fd.spielt.rolle = 'Hauptrolle' AND fd.film.titel = fd.spielt.titel;

- Each plan consists of two subplans
  - Including p_1 and p_4, even though from same source
  - Each subplan is source-specific.
    - Can be executed by the source
- Who executes the joins between the subplans?
  - Mediator
- System can select
  - All plans? (expensive, complete)
  - Fastest plan?
  - Cheapest plan?
  - Plan with the largest result?

FAQ

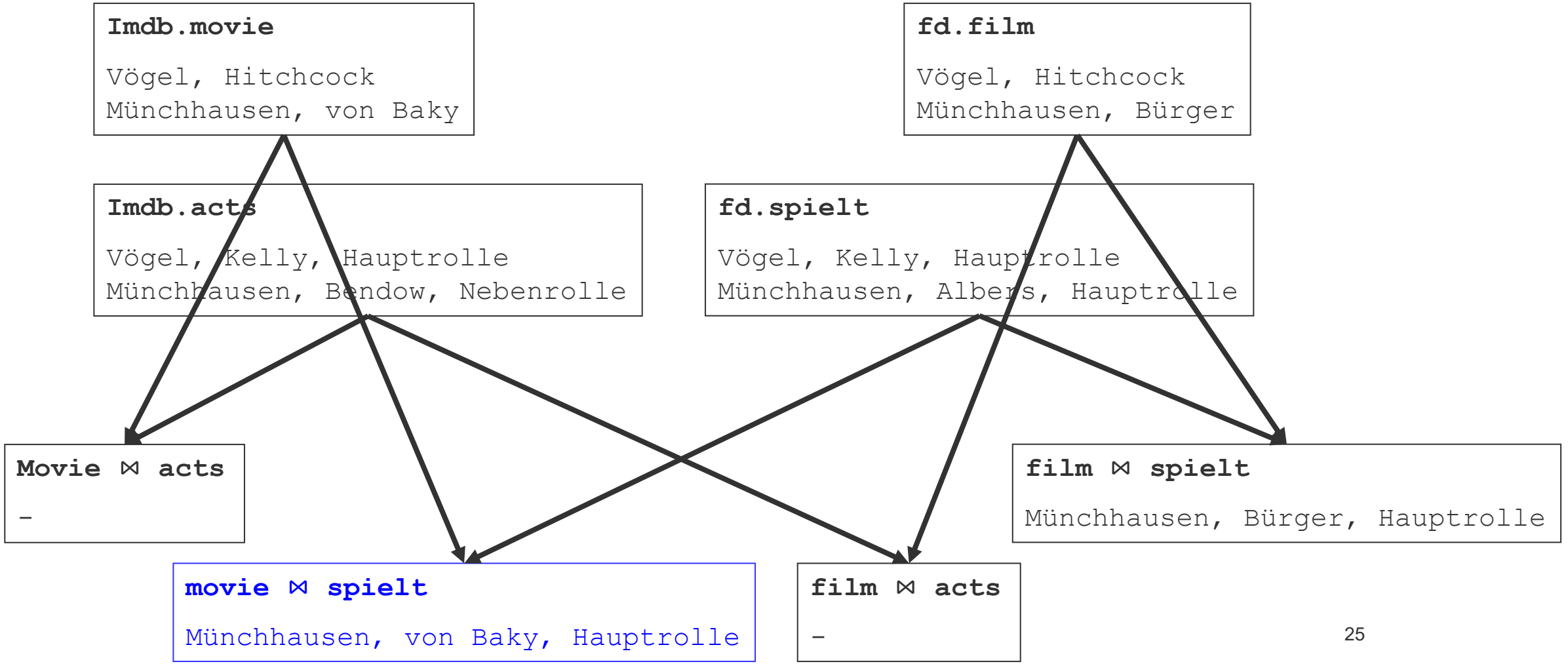
- Can one not regard **`imdb.movie ⋈ imdb.acts`** together as a single subplan?
  - Yes, but we are still assuming 1 source = 1 relation.
  - Later: GaV

- Why even use **`imdb.movie`** and **`fd.spielt`** together in one plan?
  - Because sources contain errors (missing values)
  - Combination can create new (and important!) tuples
  - Assumption: Joinable on same values in the join attributes
    - Here: movie names and actor names

- Beware of implicit consistency assumptions

Example

```
SELECT titel, regisseur, rolle
FROM   film, spielt
WHERE  spielt.schauspieler_name = 'Hans Albers'
       AND spielt.rolle = 'Hauptrolle'
       AND spielt.titel = film.titel;
```



Query Translation (Wrapper)

- Each subplan can be sent to exactly one source.
  - Translating schema element names
  - Translating a SQL query into a web service, HTTP call, XQuery, …
  - Translating constants in a query
    - What does “Hauptrolle" mean in the imdb?

```
SELECT  Title, rolle
FROM    spielt
WHERE   schauspieler_name=‚Hans Albers‘
AND     rolle=‚Hauptrolle‘
```

or

```
SELECT  title, role
FROM    imdb.acts
WHERE   actor_name=‚Hans Albers‘
AND     role=‚main role‘;
```

```
SELECT  title, role
FROM    imdb.acts
WHERE   actor_name=‚Hans Albers‘;
```

Query Optimization

- Local optimization: Each plan individually
- Global optimization: Regard the set of all plans together
- Quetions
  - Optimal execution order of the subplans?
  - Possibly parallel execution?
  - Checking selection conditions globally or locally?

```
SELECT  title, director, role
FROM    imdb.movie, imdb.acts
WHERE   imdb.acts.actor_name = 'Hans Albers'
        AND imdb.acts.role = 'main actor'
        AND imdb.acts.title = imdb.movie.title;
```

```
SP1:
SELECT  title, director
FROM    imdb.movie;
```

```
SP2:
SELECT  title, role
FROM    imdb.acts
WHERE   actor_name=,Hans Albers'
AND     role=,main role';
```

Options

```
SELECT  title, director, role
FROM    imdb.movie, imdb.acts
WHERE   imdb.acts.actor_name = 'Hans Albers'
        AND imdb.acts.role = 'main actor'
        AND imdb.acts.title = imdb.movie.title;
```

```
SP1:
SELECT  title, director
FROM    imdb.movie;
```

```
SP2:
SELECT  title, role
FROM    imdb.acts
WHERE   actor_name=‚Hans Albers‘
AND     role=‚main role‘;
```

- First SP1, then SP2, then join in the mediator
  - Requires download of all IMDB movies
  - Probably very expensive
- Better: SP1 and SP2 in parallel
  - Result of SP1 is irrelevant for SP2
  - Still rather expensive
- First SP2, then rewrite SP1, join in the mediator
  - SP2: only few tuples
  - Push these tuples to SP1 „WHERE TITLE in (…)“
    - Passing bindings
  - Parallelization no longer possible
  - Many small intermediate results, still probably much faster
- …

Query Execution

- Each of the four plans has been optimized and translated.
- Subplans must be executed
  - Send query, wait for the result, buffer locally
  - Monitor time-outs
  - Do all subplans have to be executed?
    - If we consider P1-P4 and can't push join conditions
    - Gives 8 subplans - but only 4 different subplans
    - Optimization potential
    - Either optimize globally or detect dynamically through caching
- Missing operations must be performed by the mediator.
  - Non-pushable selections
  - Joins between relations of different sources
  - sorting, aggregation, duplicate elimination, ...

Result integration

- Each plan provides a set of correct results.
- Nevertheless, there can be problems
  - Duplicates: Objects exist in multiple sources
  - Conflicts: Objects are present multiple times with conflicting information

Imdb.movie

```
Vögel, Hitchcock
Münchhausen, von Baky
```

Imdb.acts

```
Vögel, Kelly, Hauptrolle
Münchhausen, Bendow, Nebenrolle
```

fd.film

```
Vögel, Hitchcock
Münchhausen, Bürger
```

fd.spielt

```
Vögel, Kelly, Hauptrolle
Münchhausen, Albers, Hauptrolle
```

… ⋈ … **UNION** … ⋈ … **UNION** …

```
–
Münchhausen, Bürger, Hauptrolle
Münchhausen, von Baky, Hauptrolle
–
```

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. **Global as View (GaV)**
   - **Modeling**
   - Query Processing
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

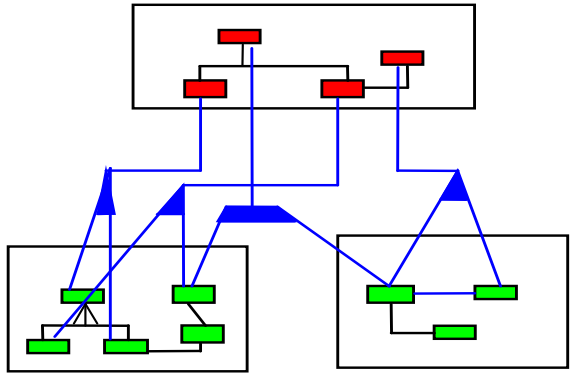
Modeling Data Sources

- Core idea
  - Modeling of structurally heterogeneous sources as views for a global schema
  - Relational model
- In general
  - A view joins several relations and produces a relation.
- Views for linking schemata
  - View defines relations of one schema and produces a relation of the other schema

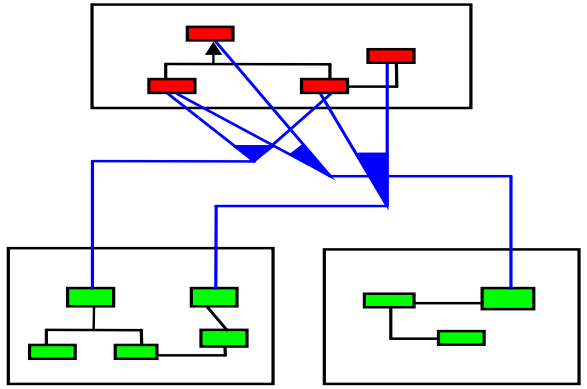
- Now: Distinguishing local and global schemata

Global as View / Local as View

- Global as View (GaV)
  - Relations of the global schema are expressed as views of the local schemata of the sources.



- Local as View (LaV)
  - Relations of the schemata of the sources are expressed as views of the global schema.



Global as View (GaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S1: IMDB(Title, Director, Year, Genre)
S2: MyMovies(Title, Director, Year, Genre)
S3: DirectorDB(Title, Director)
S4: GenreDB(Title, Year, Genre)

```
CREATE VIEW Film AS
SELECT * FROM IMDB
        UNION
SELECT * FROM MyMovies
        UNION
SELECT DirectorDB.Title, DirectorDB.Director, GenreDB.Year, GenreDB.Genre
FROM DirectorDB, GenreDB
WHERE DirectorDB.Title = GenreDB.Title
```

Typical feature of GaV

Join across multiple sources (query processing!)

Source: VL „Data Integration“, Alon Halevy, University of Washington, 2002

Global as View (GaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S5: MDB(Title, Director, Year)
S6: MovDB(Title, Director, Genre)

```
CREATE VIEW Film AS
SELECT Title, Director, Year, NULL AS Genre
FROM MDB
        UNION
SELECT Title, Director, NULL AS Year, Genre
FROM MovDB
        UNION
SELECT Title, Director, Year, Genre
FROM MDB NATURAL JOIN MovDB
```

Missing attributes

Roughly equivalent to full outer join

Source: VL „Data Integration“, Alon Halevy, University of Washington, 2002

Global as View (GaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S7: CinemaDB(Cinema, Genre)

```
CREATE VIEW Film AS
SELECT NULL, NULL, NULL, Genre
FROM CinemaDB

CREATE VIEW Program AS
SELECT Cinema, NULL, NULL
FROM CinemaDB
```

At best interesting for passing bindings

- Problem: Associations are separated
  - Since the result of a view can only be a global relation
  - Possible solution: Skolem functions (not suitable here)
- Occurs when
  - attributes of different global relations are in a single local relation and the connecting key/FK does not exist locally
  - Not in the opposite case: simply join local relations...

Source: VL „Data Integration“, Alon Halevy, University of Washington, 2002

Preview:
Local as View (LaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S7: CinemaDB(Cinema, Genre)

```
CREATE VIEW S7 AS
SELECT Program.Cinema, Film.Genre
FROM Film, Program
WHERE Film.Title = Program.Title
```

- Inverted view
- Associations remain
  - Title is never instantiated,
  - but you know that there exists a connecting title.

Integrity Constraints (ICs)

- Restrict value ranges, exclude values, exclude combinations of values, ...
- Serve to determine the intension of a relation in more detail:
  - `expensive_cars( auto, price ), price>50.000`
  - `golfclub_members( name, age ), age>55`
  - `film( title, type ), type∈{drama, docu, short}`
  - …
- Important for integration

Felix Naumann
Data Integration
Summer 2025

Global as View (GaV) – Gobal ICs

Global schema
NewFilm(Title, Director, Year, Genre)
IC: Year > 2000
Program(Cinema, Title, Time)

```
CREATE VIEW NewFilm AS
SELECT * FROM IMDB
WHERE Year > 2000
            UNION
SELECT * FROM MyMovies
WHERE Year > 2000
```

S1: IMDB(Title, Director, Year, Genre)
S2: MyMovies(Title, Director, Year, Genre)

- IC on global schema can be included in the view definition in this case
- Execution in the source (push) or in the mediator

Problems with global ICs

Global schema

NewFilms(Title, Director, Year, Genre);

IC: Year > 2000

Program(Cinema, Title, Time);

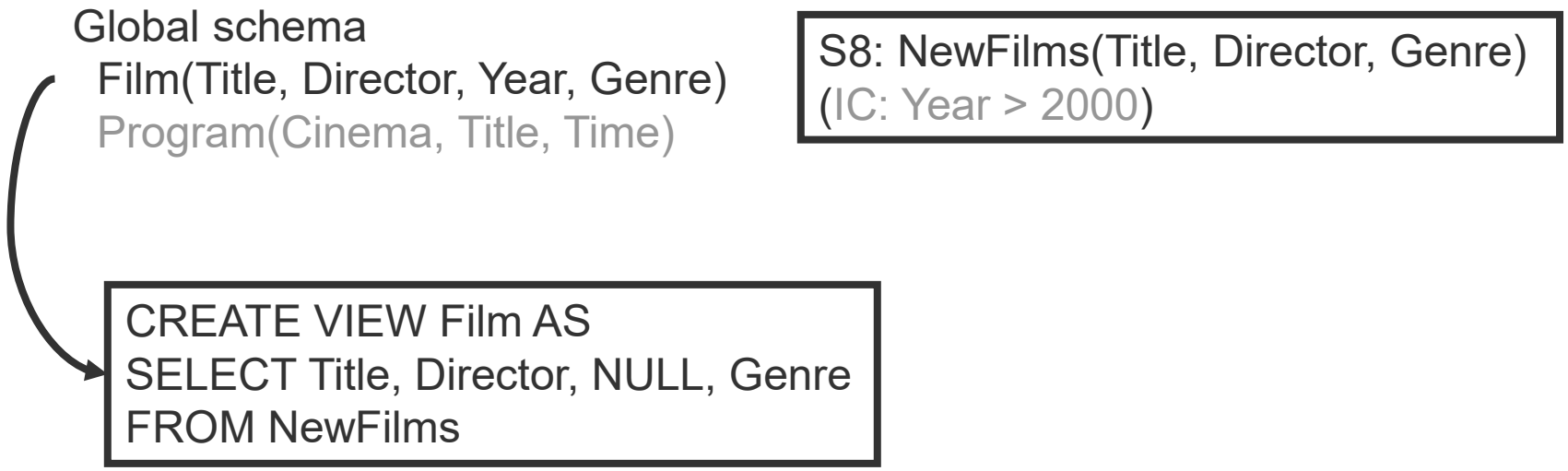
```
S1: IMDB( Title, Director, Year, Genre)
S2: MyOldOrNewMovies( Title, Director)
```

```
CREATE VIEW NewFilms AS
SELECT  *
FROM    IMDB
WHERE   Year > 2000
        UNION
SELECT Title, Director, NULL, NULL
FROM    MyOldOrNewMovies
WHERE   ??
```

- Source does not give any year information
- Unable to validate IC on global schema
- Unable to integrate source

Felix Naumann
Data Integration
Summer 2025

Global as View (GaV) – Local ICs



- Known constraints on source cannot be modeled.
- Why is that a pity?
  - `SELECT * FROM film WHERE Year < 1950`

„Trick" if attributes are exported

Global schema

```
Film( Title, Director, Year, Genre)
Program( Cinema, Title, Time)
```

```
S8: NewFilms( Title, Director, Year, Genre)
IC: Year > 2000
```

```
CREATE VIEW Film AS
SELECT  Title, Director, Year, Genre
FROM    NewFilms
WHERE   Year > 2000
```

- IC of source is modelled
  - (Only works if the attribute is instantiated in the source)
- The condition does not change the result
  - It would be pointless for a local query
- But it helps the query planner
  - `SELECT * FROM Film WHERE Year < 1950`

Global as View (GaV) – ICs

Summary

- Constraints in the global schema
  - can prevent the integration of sources, if the condition cannot be checked (missing attribute).

- Constraints in local schemata
  - can be modeled,
    - (if they apply to exported attributes, or)
    - if they apply to global attributes and have the form attribute = <constant> .
  - may not contribute to the query result,
    - if constraint has not been modeled.

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. **Global as View (GaV)**
   - Modeling
   - **Query Processing**
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

Query processing – GaV

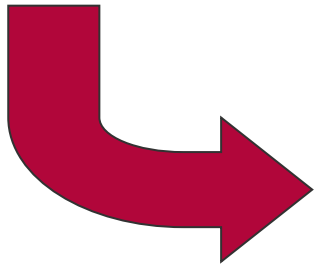
- Given:
  - Query against global schema
    - In particular: on relations of the global schema
  - For each global relation, exactly one view on local sources
- Find:
  - All tuples that meet the query conditions
  - However, data is stored in local sources.
- Idea:
  - Replace every relation of the query with its view definition-
    - View unfolding, view expansion, query unfolding, …
  - Result: nested query

GaV – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S1: IMDB(Title, Director, Year, Genre)
S2: DirectorDB(Title, Director)
S3: GenreDB(Title, Year, Genre)

```
SELECT *
FROM Film
```



Global view for film

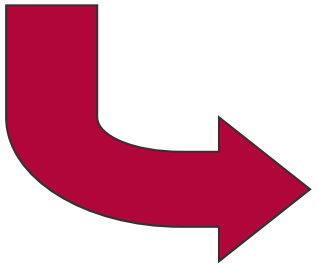
```
SELECT *
FROM (      SELECT * FROM IMDB
                  UNION
            SELECT R.Title, R.Director, G.Year, G.Genre
            FROM DirectorDB R, GenreDB G
            WHERE R.Title = G.Title
      )
```

GaV – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S1: IMDB(Title, Director, Year, Genre)
S2: DirectorDB(Title, Director)
S3: GenreDB(Title, Year, Genre)

```
SELECT Title, Year
FROM Film
WHERE Year = ‚2003‘
```



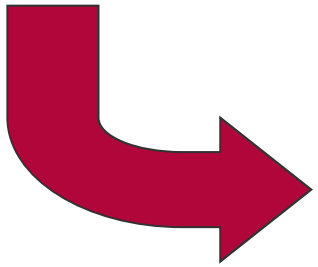
```
SELECT Title, Year
FROM (      SELECT * FROM IMDB
                  UNION
            SELECT R.Title, R.Director, G.Year, G.Genre
            FROM DirectorDB R, GenreDB G
            WHERE R.Title = G.Title
      )
WHERE Year = ‚2003‘
```

Felix Naumann
Data Integration
Summer 2025

GaV – Example

```
SELECT F.Title, P.Cinema
FROM   Film F, Program P
WHERE F.Title = P.Title
AND      P.Time > 20:00
```

```
S1: IMDB(Title, Director, Year, Genre)
S2: DirectorDB(Title, Director)
S3: GenreDB(Title, Year, Genre)
S7: CinemaDB(Title, Cinema, Genre, Time)
```



```
SELECT F.Title, P.Cinema
FROM (      SELECT * FROM IMDB
                        UNION
                  SELECT R.Title, R.Director, G.Year, G.Genre
                  FROM DirectorDB R, GenreDB G
                  WHERE R.Title = G.Title
           ) AS F,
           (      SELECT *
                   FROM CinemaDB
           ) AS P
WHERE F.Title = P.Title
AND       P.Time > 20:00
```

Felix Naumann
Data Integration
Summer 2025

GaV - Nesting

- Nested queries
- Arbitrarily deep nesting
  - Processing
    - Conceptually:
      Execute queries from the inside to the outside; storage in temporary relations.
    - In practice:
      Optimization potential by rewriting the query (unnesting)
  - Caching and materialized views

GaV - Nesting

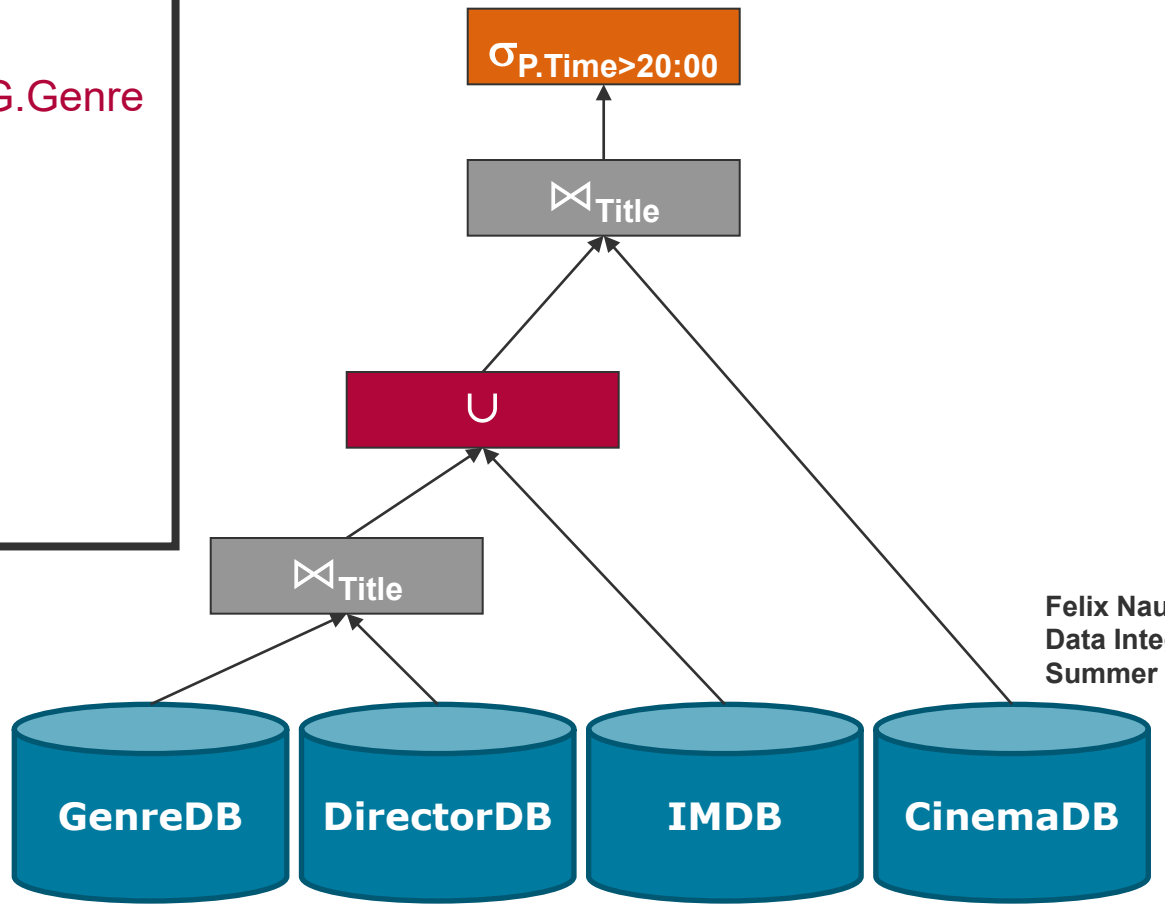
- Nesting
  - Data Integation: internal queries are queries to sources
  - Materialized views: inner queries and outer queries are queries to base relations.

```
SELECT F.Title, P.Cinema
FROM (      SELECT * FROM IMDB
                    UNION
              SELECT R.Title, R.Director, G.Year, G.Genre
              FROM DirectorDB R, GenreDB G
              WHERE R.Title = G.Title
       ) AS F,
       (      SELECT *
              FROM CinemaDB
       ) AS P
WHERE F.Title = P.Title
AND       P.Time > 20:00
```

```
SELECT F.Title, P.Cinema
FROM     IMDB F, CinemaDB P
WHERE F.Title = P.Title
AND       P.Time > 20:00

UNION

SELECT F1.Title, P1.Kino
FROM     DirectorDB F1, GenreDB F2, CinemaDB P1
WHERE F1.Title = F2.Title
WHERE F1.Title = P1.Title
AND       P1.Time > 20:00
```

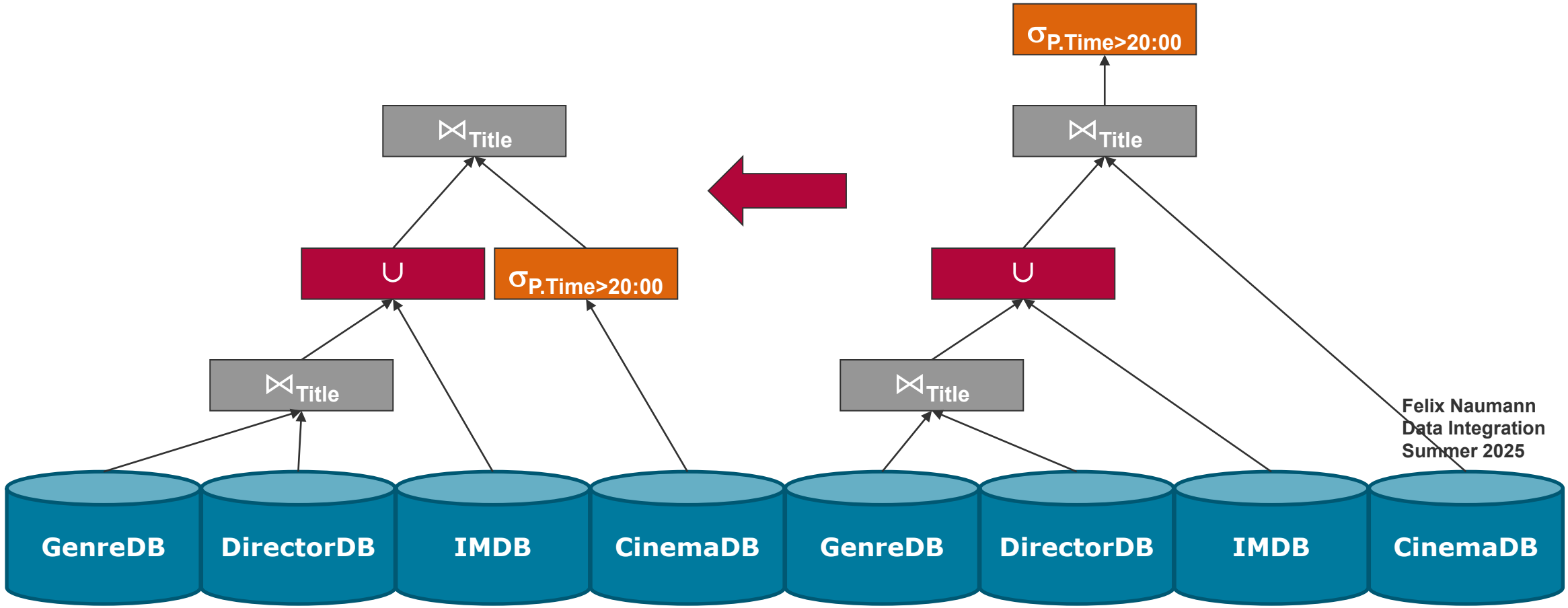
GaV - Nesting

```
SELECT F.Title, P.Cinema
FROM (      SELECT * FROM IMDB
                        UNION
                SELECT R.Title, R.Director, G.Year, G.Genre
                FROM DirectorDB R, GenreDB G
                WHERE R.Title = G.Title
          ) AS F,
          (      SELECT *
                 FROM CinemaDB
          ) AS P
WHERE F.Title = P.Title
AND       P.Time > 20:00
```

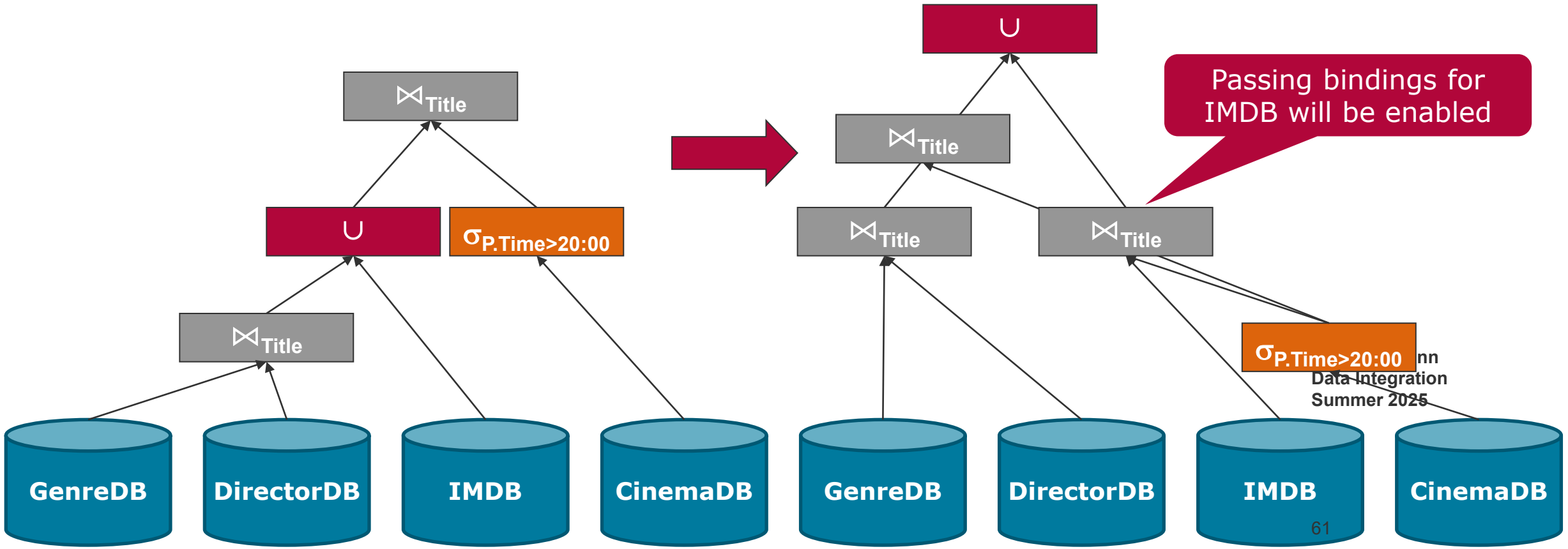


Felix Naumann
Data Integration
Summer 2025

GaV - Nesting


⋈Title
∪
σP.Time>20:00
⋈Title
GenreDB
DirectorDB
IMDB
CinemaDB
σP.Time>20:00
⋈Title
∪
⋈Title
GenreDB
DirectorDB
IMDB
CinemaDB

GaV - Nesting


⋈Title
∪
σP.Time>20:00
⋈Title
GenreDB
DirectorDB
IMDB
CinemaDB
∪
⋈Title
⋈Title
⋈Title
Passing bindings for IMDB will be enabled
σP.Time>20:00
GenreDB
DirectorDB
IMDB
CinemaDB

GaV - Nesting

Global schema
Film(Title, Director, Genre)
Program(Cinema, Title, Time)

S5: MDB(Title, Time, Director)
S6: MovDB(Cinema, Genre, Title)

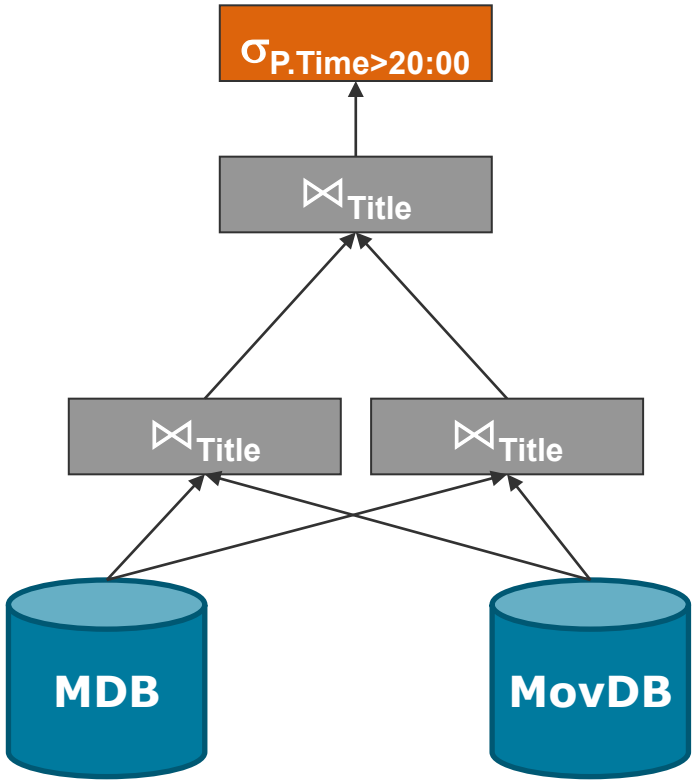
```
CREATE VIEW Film AS
SELECT Title, Director, Genre
FROM MDB, MovDB
WHERE MDB.Title = MovDB.Title

CREATE VIEW Program AS
SELECT Cinema, Title, Time
FROM MovDB, MDB
WHERE MDB.Title = MovDB.Title
```

```
SELECT F.Title, P.Cinema
FROM   Film F, Program P
WHERE F.Title = P.Title
AND      P.Time > 20:00
```

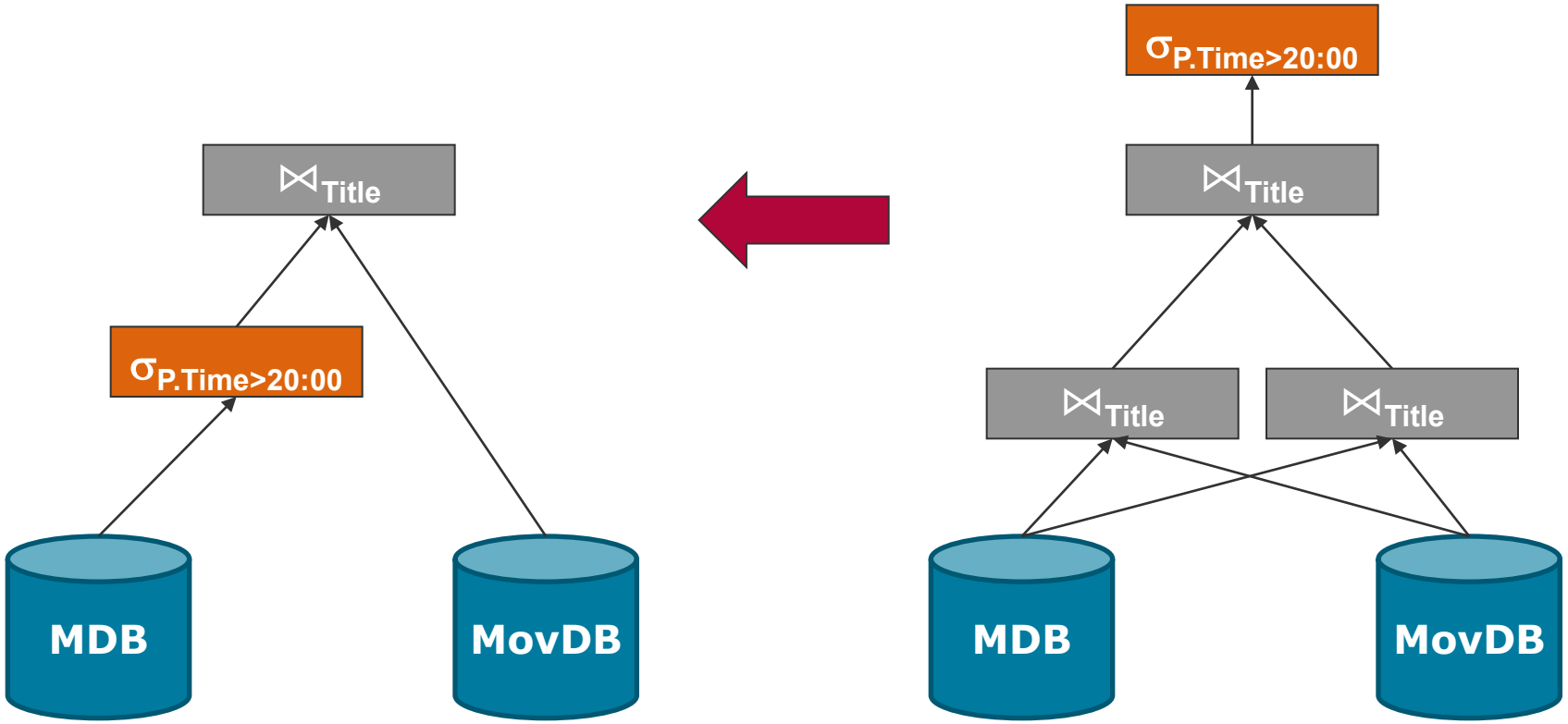
```
SELECT F.Title, F.Time
FROM (   SELECT Title, Director, Genre
         FROM MDB, MovDB
         WHERE MDB.Title = MovDB.Title
      ) AS F,
      (   SELECT Cinema, Title, Time
         FROM MovDB, MDB
         WHERE MDB.Title = MovDB.Title
      ) AS P
WHERE F.Title = P.Title
AND      P.Time > 20:00
```

GaV - Nesting

```
SELECT F.Title, F.Time
FROM (  SELECT Title, Director, Genre
        FROM MDB, MovDB
        WHERE MDB.Title = MovDB.Title
      ) AS F,
      (  SELECT Cinema, Title, Time
        FROM MovDB, MDB
        WHERE MDB.Title = MovDB.Title
      ) AS P
WHERE F.Title = P.Title
AND     P.Time > 20:00
```

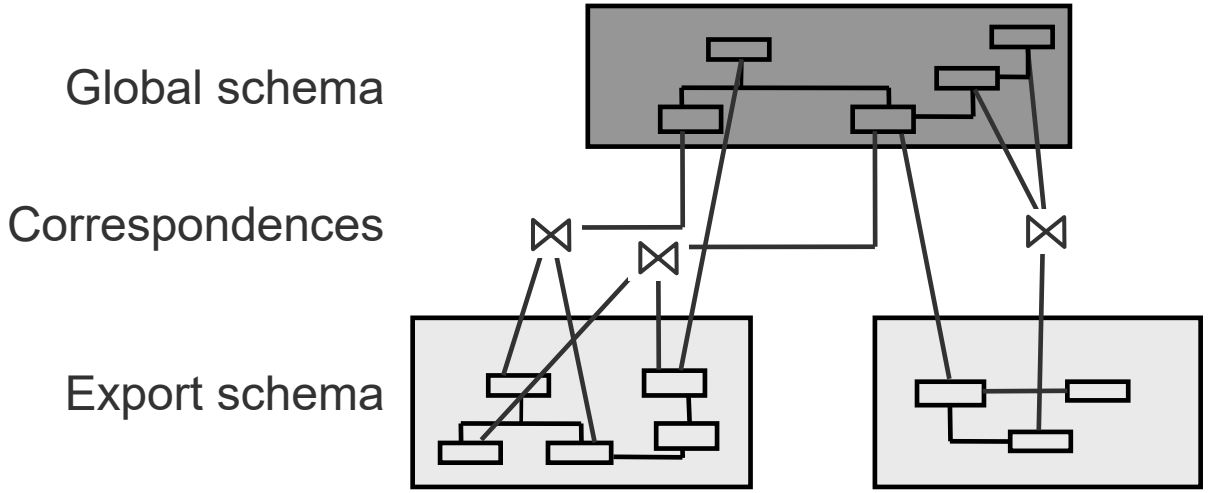


GaV - Nesting



Felix Naumann
Data Integration
Summer 2025

Summary – GaV



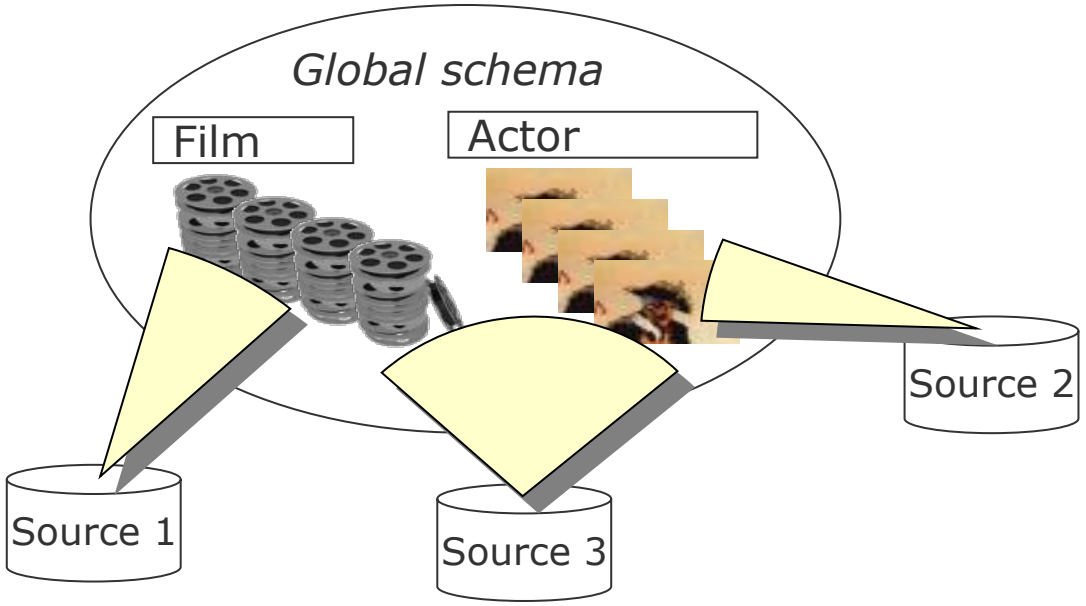
- Task of query processing
  - Given a query *q* against the global schema
  - Given a set of correspondences/logical mappings between global and local schemata
  - Find all the results for *q*.

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. **Local as View (LaV)**
   - **Modeling**
   - Applications
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

Why LaV? A different point of view

- In the world, there are many films, actors, …
- The global schema models this world.
- Theoretically, the extension is thus fixed.
  - But no one knows it.
  - Information integration attempts to create it.
- Sources store views of the global extension.
  - In other words, excerpts of the real world
- We can only use those.



Local as View (LaV) – Example

Global schema
  Film(Title, Director, Year, Genre)
  Program(Cinema, Title, Time)

S1: IMDB(Title, Director, Year, Genre)
S2: MyMovies(Title, Director, Year, Genre)
S3: DirectorDB(Title, Director)
S4: GenreDB(Title, Year, Genre)

```
CREATE VIEW S1 AS
SELECT * FROM Film

CREATE VIEW S2 AS
SELECT * FROM Film

CREATE VIEW S3 AS
SELECT Film.Title, Film.Director
FROM Film

CREATE VIEW S4 AS
SELECT Film.Title, Film.Year,
            Film.Genre
FROM Film
```


Source: VL „Data Integration“, Alon Halevy, University of Washington, 2002

Local as View (LaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S9: ActorDB(Title, Schauspieler, Year)

„Missed opportunity“

```
CREATE VIEW S9 AS
SELECT Title, NULL, Year
FROM Film
```

Local as View (LaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S7: CinemaDB(Cinema, Genre)

```
CREATE VIEW S7 AS
SELECT Program.Cinema, Film.Genre
FROM Film, Program
WHERE Film.Title = Program.Title
```

- Associations of the global schema can be modelled in the view.

Local as View (LaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S9: Films(Title, Year, Ort, DirectorID)
Director(ID, Name)

```
CREATE VIEW S9Films AS
SELECT Title, Year, NULL, NULL
FROM Film

CREATE VIEW S9Director AS
SELECT NULL, Director
FROM Film
```

- Associations of the local schema cannot be modelled.

Local as View (LaV) – Example

Global schema
Film(Title, Director, Year, Genre)
Program(Cinema, Title, Time)

S8: NewFilms(Title, Director, Genre)
(IC: Year > 2000)

```
CREATE VIEW S8 AS
SELECT Title, Director, Genre
FROM Film
WHERE Year > 2000
```

- IC on source can be modeled
  - If the attribute exists in the global schema
- ICs do not need to be explicitly defined in the source
  - Implicit constraints can also be included in the view.

Local as View (LaV) – Global ICs

Global schema
NewFilm(Title, Director, Year, Genre)
Program(Cinema, Title, Time)
Constraint: Year > 2000

S1: IMDB(Title, Director, Year, Genre)
S2: MyMovies(Title, Director, Year, Genre)

```
CREATE VIEW S1 AS
SELECT * FROM NewFilm
(WHERE Year> 2000)

CREATE VIEW S2 AS
SELECT * FROM NewFilm
(WHERE Year> 2000)
```

- We cannot well model constraints on the global schema.
  - Was possible with GaV

Local as View (LaV) – local ICs

Global schema
Film(Title, Director, Year, Genre)

S1: AllFilmsNice(Title, Director, Year, Genre)
S2: AllFilmsEvil(Title, Director, Genre)
S3: NewFilmsNice(Title, Director, Year, Genre)
(constraint: Year > 2000)
S4: NewFilmsEvil(Title, Director, Genre)
(constraint: Year > 2000)
S5: NewFilms(Title, Director, Genre)
(constraint: Year = 2020)

```
CREATE VIEW S1 AS
SELECT * FROM Film

CREATE VIEW S2 AS
SELECT Title, Director, Genre
FROM Film

CREATE VIEW S3 AS
SELECT * FROM Film
(WHERE Year > 2000)

CREATE VIEW S4 AS
SELECT Title, Director, Genre
FROM Film
WHERE Year > 2000

CREATE VIEW S5 AS
SELECT Title, Director, Genre
FROM Film
WHERE Year = 2020
```

Modelling for optimization

Modelling for answerability

Local ICs: More examples

Datenquelle	Beschreibung
`spielfilme(titel, regisseur, laenge)`	Informationen über Spielfilme, die mindestens 80 Minuten Länge haben.
`kurzfilme(titel, regisseur)`	Informationen über Kurzfilme. Kurzfilme sind höchstens 10 Minuten lang.
`filmkritiken(titel, regisseur, schauspieler, kritik)`	Kritiken zu Hauptdarstellern von Filmen
`us_spielfilme(titel, laenge, schauspieler_name)`	Spielfilme mit US-amerikanischen Schauspielern
`spielfilm_kritiken(titel, rolle, kritik)`	Kritiken zu Rollen in Spielfilmen
`kurzfilm_rollen(titel, rolle, schauspieler_name, nationalitaet)`	Rollenbesetzungen in Kurzfilmen

```
film(titel,typ,regisseur,laenge);
schauspieler(schauspieler_name,nationalitaet);
spielt(titel,schauspieler_name,rolle,kritik);
```

```
film(T,Y,R,L),L>79,Y='Spielfilm'                ⊇  spielfilme(T,R,L)
film(T,Y,R,L),L<11,Y='Kurzfilm'                 ⊇  kurzfilme(T,R)
film(T,_,R,_),spielt(T,S,O,K),O='Hauprolle'     ⊇  filmkritiken(T,R,S,K)
film(T,Y,_,L),spielt(T,S,_,_),
schauspieler(S,N), N='US',Y='Spielfilm'         ⊇  us_spielfilm(T,L,S)
film(T,Y,_,_),spielt(T,_,O,K),Y='Spielfilm'     ⊇  spielfilm_kritiken(T,O,K)
film(T,Y,_,_),spielt(T,S,O,_),
schauspieler(S,N), Y='Kurzfilm'                 ⊇  kurzfilm_rollen(T,O,S,N)
```

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. **Local as View (LaV)**
   - Modeling
   - **Applications**
   - Query Processing
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

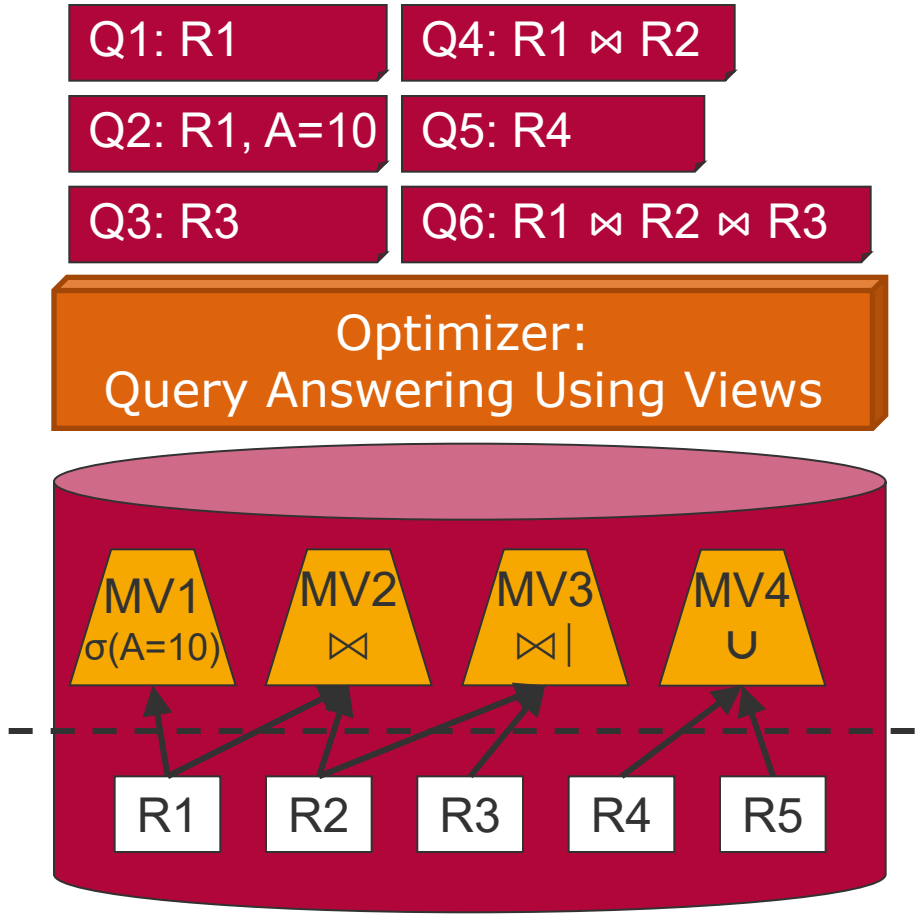
Felix Naumann
Data Integration
Summer 2025

LaV – Applications

- Query optimization
  - Materialized views on database schemata
- Data warehouse design
  - Materialized views on warehouse schema
- Semantic caching
  - Materialized data at the client
- Data integration
  - Data sources as views on global (mediator) schema

LaV Application: Query Optimization

- Materialized views (MV) on database schema
  - MQT: Materialized Query Table
  - AST: Advanced Summary Table
- Which views help to answer a database query by precalculating predicates?
- Problems
  - It's not always better to use an MV (indexes!).
  - Updating ofMVs
    - Write on MV
    - Write on base relations



Felix Naumann
Data Integration
Summer 2025

LaV Application: Data Warehouse Design

- Views on warehouse schema

- Give a query workload
  - Query workload = set of queries plus their frequencies
- Which views should I materialize?
  - To respond to all queries from the workload

- More general:
  - Given a query workload, which views should I materialize to answer the workload optimally.
    - Idea: Check all combinations
  - Question: Why is this problem actually quite simple?
    - Better: Include costs – disk space, view updates, indexes

LaV Application: Semantic Caching

- In distributed DBMS
- Materialized data at the client
  - Originate from (complex) queries

- Given a query
  - What data in the cache can I use to answer it?
  - What new data do I have to query fresh?

- Also: Which views should I pre-calculate?

LaV Application: Data Integration

- Data sources as views on global (mediator) schema

- Questions:
  - How can I answer a query to the global schema using only the views?
    - Difference to query optimization: no base tables available
  - Can I answer the query completely (extensionally)?

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. **Local as View (LaV)**
   - Modeling
   - Applications
   - **Query Processing**
   - Containment
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

Query Planning

- Given
  - A query q to the global schema
  - Local schemata

- Find
  - Sequence of queries $q_1 \diamond \ldots \diamond q_n$
  - Each q_i can be executed by a wrapper
  - The appropriate linking of $q_1,\ldots,q_n$ answers q
    - Within a plan through joins: $\diamond \rightarrow \bowtie$
    - Across plans through UNION: $\diamond \rightarrow \cup$
  - Tuples determined by $q_1 \bowtie \ldots \bowtie q_n$ are correct answers to q.

Query Plan

- We call $q_1 \bowtie \ldots \bowtie q_n$ a query plan.
- Definition
  Given a global query q. A query plan p for q is a query of the form $q_1 \bowtie \ldots \bowtie q_n$ so that
  - each q_i can be executed by exactly one wrapper,
  - and each tuple computed by p is a semantically correct answer for q.

- Comments
  - We have not yet defined "semantically correct".
  - Usually, there are many query plans.
  - The q_i are calls subqueries or subplans.

Query Containment

- Intuitively, we want the following:
  - A view v returns (only) semantically correct queries to a global query q if its extension is included in the result of q.
- Definition
  - Let S be a database schema, I an instance of S, and q1, q2 queries against I.
  - Let q(I) be the result of a query q on I.
  - Then:

    q1 contained in q2 ($q1 \subseteq q2$)

    $\Leftrightarrow$

    $q1(I) \subseteq q2(I)$ for any instance I
- Comments
  - The important part is the last one: "for any instance I of S"
  - Of course, we cannot try them all.

Semantic Correctness

- This allows us to define when a plan is semantically correct.
  - But we cannot test that yet.

- Definition
  - Let S be a global schema, q a query against S,
  - and p a linking of views v1,...,vn against S, which are defined as the left side of LaV correspondences.
  - Then: p is semantically correct for q $\Leftrightarrow$ p $\subseteq$ q

- Comments
  - Thus, the extension of p is contained in that of q.
  - Of course, the extension of q does not exist (only virtually)

Many Query Plans

- Definition
  Given a global query q. Let p_1, …, p_n be the set of all semantically correct query plans for q. Then the result of q is defined as

$$result(q) = \bigcup_{i=1..n} result(p_i)$$

- Comments
  - The UNION operator removes duplicates.
    - Behind this lies the problem of integrating result tuples.
  - How the result is calculated is a matter of query optimization.
    - Plans can overlap in partial queries.
  - The result of q depends on the defined correspondences.

Answering Queries using Views

- Idea
  - Query rewriting by incorporating the views
  - Cleverly combine the individual views into a query so that their result answers part of the query (or the whole query).
  - The total result is then the UNION of the results of several query rewritings.

LaV – Example

Global schema

Teaches(prof,course_id, sem, eval, univ)
Course(course_id, title, univ)

Source 1: All database courses

```
CREATE VIEW DB-course AS
SELECT C.Title, L.prof, C.class_id, C.univ
FROM    Teaches L, Course C
WHERE  L.course_id = C.course_id
AND      L.univ = C.univ
AND      C.title LIKE „%_databases“
```

Source 2: All HPI-lectures

```
CREATE VIEW HPI-VL AS
SELECT C.Title, L.prof, C.course_id, C.univ
FROM    Teaches L, Course C
WHERE L.course_id = C.course_id
AND      C.univ = „HPI“
AND       L.univ =  „HPI“
AND      C.title LIKE „%VL_%“
```

Global query

```
SELECT prof
FROM Teaches L, Course C
WHERE L.course_id = C.course_id
AND C.title LIKE „%_ databases“
AND L.univ = „HPI“
```

Rewritten query

```
SELECT prof
FROM DB-course D
WHERE D.univ = „HPI“
```

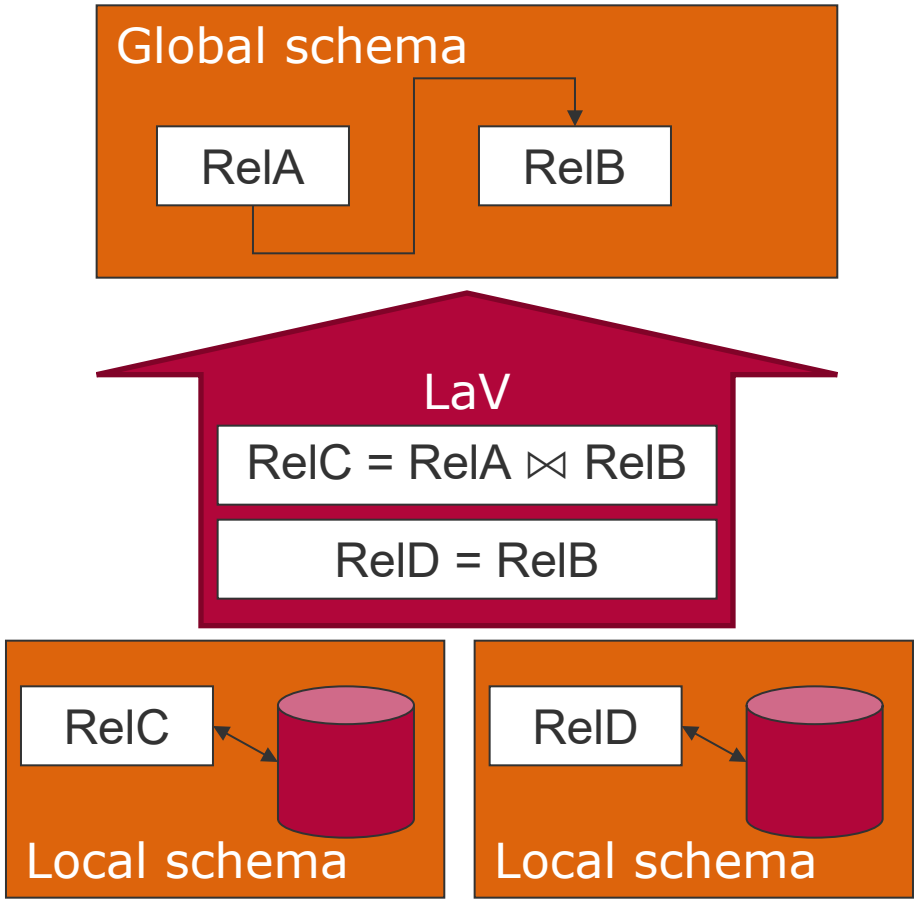
Question: Why not include Source 2?

Answer: Because Source 1 already delivers **all** DB courses.

Closed world assumption

Felix Naumann
Data Integration
Summer 2025

LaV Visualization (OWA)



User query

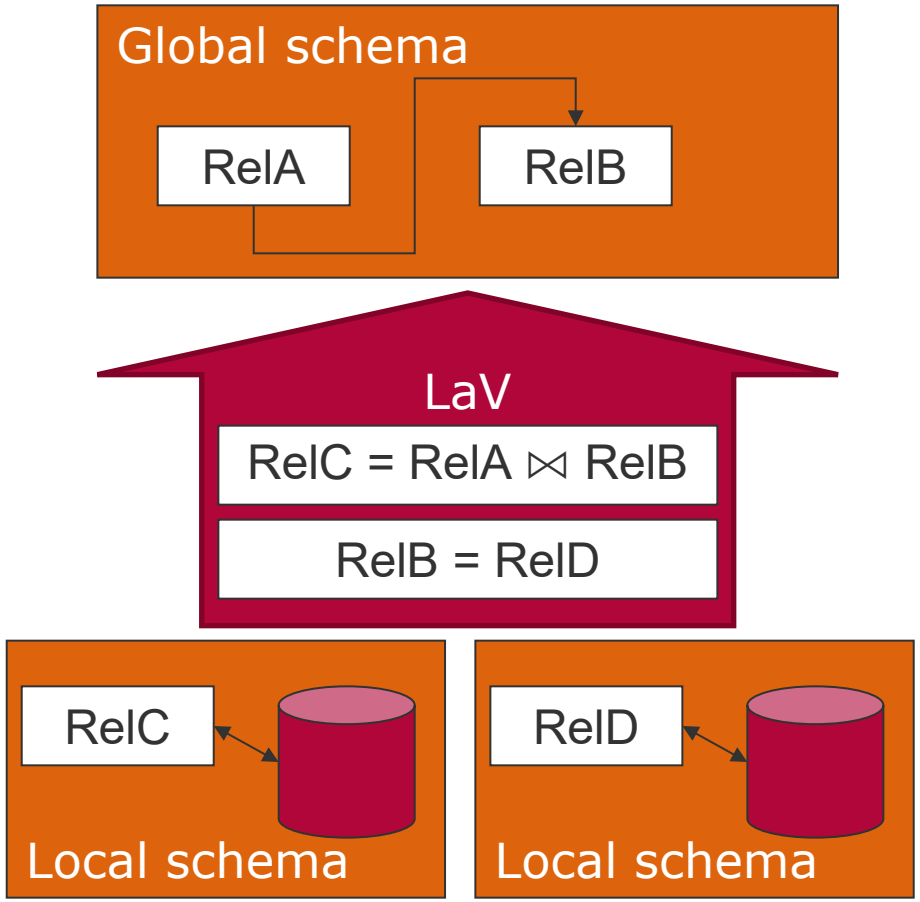
```
SELECT  ???
FROM    RelB
WHERE   ???
```

Query rewriting

Rewritten query

```
SELECT  ???
FROM    RelD
WHERE   ???
UNION
SELECT  Attr(B)
FROM    RelC
WHERE   ???
```

LaV Visualization (CWA)



User query

```
SELECT ???
FROM   RelA NATURAL JOIN RelB
WHERE  ???
```

Query rewriting

Rewritten query

```
SELECT ???
FROM   RelC
WHERE  ???
```

LaV – Example

Subset of global schema

Teaches(prof,course_id, sem, eval, univ)
Course(course_id, title, univ)

Source 1: All databases courses

```
CREATE VIEW DB-course AS
SELECT C.title, L.prof, C.course_id, C.univ
FROM    Teaches L, Course C
WHERE L.course_id = K.course_id
AND      L.univ = C.univ
AND      C.title LIKE „%_Datenbanken“
```

Source 2: All HPI lectures

```
CREATE VIEW HPI-VL AS
SELECT C.title, L.prof, C.course_id, C.univ
FROM    Teaches L, Course C
WHERE L.course_id = C.course_id
AND      C.univ = „HPI“
AND      L.univ =  „HPI“
AND      C.title LIKE „%VL_%“
```

Global query

```
SELECT Title, course_id
FROM Course C
WHERE C.univ = „HPI“
```

Rewritten query

```
SELECT title, course_id
FROM DB-course D
WHERE D.univ = „HPI“
          UNION
SELECT title, course_id
FROM HPI-VL
```

Question: Why include Source 2 here after all?

LaV – Beispiel Vergleich

Global query

```
SELECT prof
FROM Teaches L, Course C
WHERE L.course_id = C.course_id
AND C.title = „VL_Datenbanken“
AND L.univ = „HPI“
```

Rewritten query

```
SELECT prof
FROM DB-course D
WHERE D.univ = „HPI“
```

} Complete result (CWA)

Global query

```
SELECT title, course_id
FROM Course C
WHERE L.univ = „HPI“
```

Rewritten query

```
SELECT title, course_id
FROM DB-course D
WHERE D.univ = „HPI“
        UNION
SELECT title, course_id
FROM HPI-VL
```

} Maximal result

Question:
What is missing?

Felix Naumann
Data Integration
Summer 2025

CWA / OWA

- Closed World Assumption (CWA)
  - Union of all data of the base relations corresponds to all relevant data.
  - Examples: Data Warehouse (and traditional DBMS)

- Open World Assumption (OWA)
  - Union of all data of the data sources is a subset of all relevant data.
  - Problems
    - Global relation content is not fixed: Query results are subject to change
    - Definition of completeness of results: What part of the *world* does the result have?
    - Negation in queries

CWA / OWA – Example

- Relation R(A,B)
- View 1
  - CREATE VIEW V1 AS
    SELECT A FROM R
  - Extension: a
- View 2
  - CREATE VIEW V2 AS
    SELECT B FROM R
  - Extension: b
- Query: SELECT * FROM R
  - CWA: (a,b) must be in the extension of R.
    - If there is any other (a, x), x would be in the extension of V2.
  - OWA: (a,b) does not have to be in the extension of R.

R = (a,b)

R = (a,b)
or
R = (a,x)
(y,b)

LaV – Query Processing

- Given: query Q and views V_1, ..., V_n
- Find: rewritten query Q‘, which
  - For optimization: is equivalent (Q = Q‘).
  - For integration: is maximally contained
    - I.e., $Q \supseteq Q‘$ and there is no Q‘‘ with $Q \supseteq Q‘‘ \supset Q‘$

- Problem:
  - How do you define and test equivalence and maximal containment?

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. **Local as View (LaV)**
   - Modeling
   - Applications
   - Query Processing
   - **Containment**
   - Bucket Algorithm
6. Global Local as View (GLaV)
7. Comparison

Felix Naumann
Data Integration
Summer 2025

LaV – Query Rewriting

- Given
  - Query Q
  - View V (view) (or plan)
- Questions
  - Is the result of V identical to the result of Q?
  - In short: Is V equivalent to Q (V = Q)?

- Reduce to a statement about *containment*
  - Is the result of V included in Q?
  - In short: Is V contained in Q ($V \subseteq Q$)?

- Then we can derive equivalence
  - $V \subseteq Q$, $Q \subseteq V \Rightarrow V = Q$

LaV – Query Rewriting (reminder)

- Query containment
  - Let S be a schema. Let Q and Q' be queries against S.
  - An instance of S is any database D with schema S.
  - The result of a query Q against S on a database D, written as Q(D), is the set of all tuples that the execution of Q in D yields.
  - Q‘ is contained in Q (Q‘ $\subseteq$ Q) $\Leftrightarrow$ Q‘(D) $\subseteq$ Q(D) <u>for any D</u>.

```
SELECT C.title, L.prof, C.course_id, C.univ
FROM   Teaches L, Course C
WHERE L.class_id = C.course_id
AND      C.univ = „Humboldt“
AND      L.univ =  „Humboldt“
AND      C.title LIKE „%VL_%“
```

$\subseteq$

```
SELECT C.title, L.prof, C.class_id, C.univ
FROM   Teaches L, Course C
WHERE L. course_id = C. course_id
AND      C.univ = „Humboldt“
AND      L.univ =  „Humboldt“
```

LaV – Examples

```
SELECT C.title, C.course_id
FROM   Course C
AND    C.univ = „Humboldt“
AND    C.title LIKE „%VL_%“
```

≠

```
SELECT C.title, C.univ
FROM   Course C
AND    C.univ = „Humboldt“
AND    C.title LIKE „%VL_%“
```

```
SELECT C.title, C.course_id
FROM   Course C
AND    C.univ = „Humboldt“
```

≠

```
SELECT C.title
FROM   Course C
AND    C.univ = „Humboldt“
```

LaV – Examples

```
SELECT C.title, C.univ
FROM    Teaches L, Course C
WHERE L.course_id = C.course_id
AND       C.univ = „Humboldt“
AND       L.univ =  „Humboldt“
```

⋂

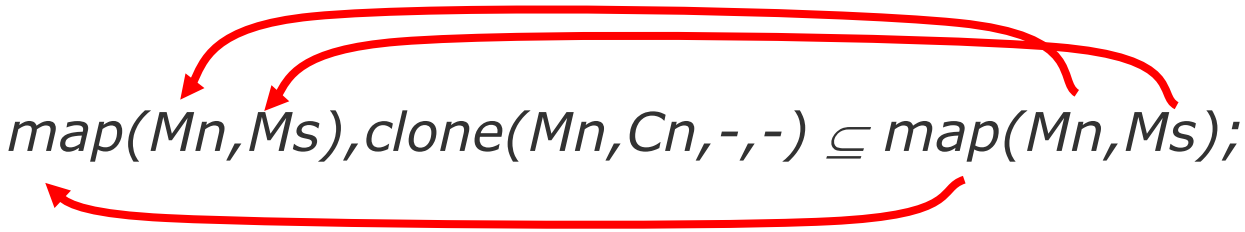
```
SELECT  C.title, C.univ
 FROM    Course C
 AND       C.univ = „Humboldt“
```

- Checking containment by checking all possible databases?
  - Too complex!
- Checking containment by finding a *containment mapping*
  - NP-complete in |Q|+|Q‘| by [CM77]
  - Multiple algorithms

Containment Mapping

- $p \subseteq u \Leftrightarrow$ there is a *containment mapping* from u to p

- Containment mapping:
  - h: sym(u) → sym(p) (mapping of symbols)
  - CM1: Each constant in u is mapped to the same constant in p.
  - CM2: Each exported variable in u is mapped to an exported variable in p.
  - CM3: Each literal (relation) in u is mapped to at least one literal in p.
  - CM4: The conditions of p imply the conditions of u.
- Proof: see [CM77]



Finding Containment Mappings

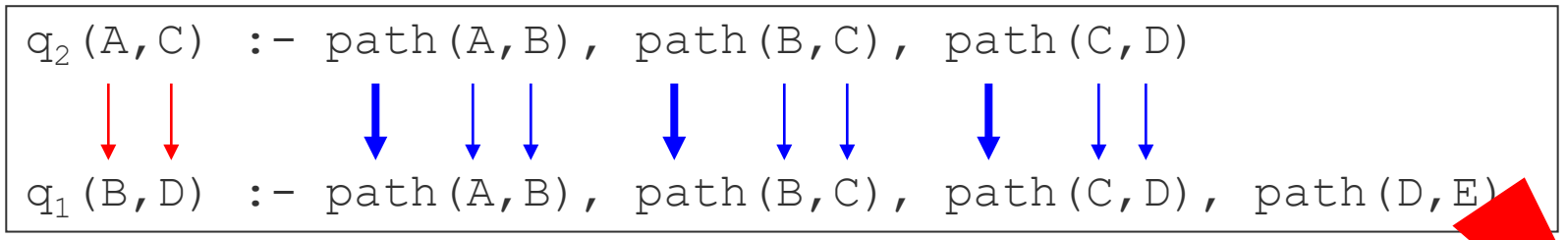
- Original motivation: Query minimization
  - Preparation step for query optimization
- Problem is NP-complete
  - Search space is exponential in the number of literals
  - Proof: Reduction to „Exact Cover“
- Thus: Try all possibilities
  - Create a search tree
    - Each level corresponds to a literal
    - Fan out according to all possible CMs
- Algorithm not here

Further Containment Examples

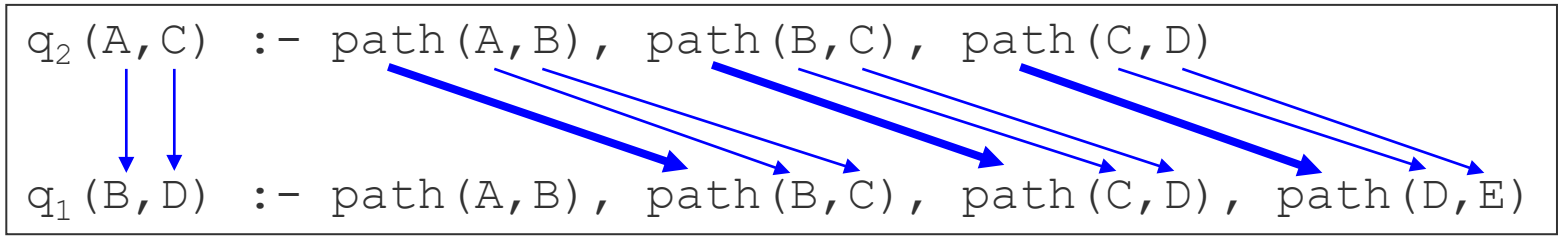
- product(PID, PN, PGID, PGN), PGN=`Wasser´ ⊆ product(PID, PN, PGID, PGN)
- product(PID, PN, PGID, PGN) ⊈ product(PID, PN, PGID, PGN), PGN=`water´
- product(PID, PN, PGID, PGN) ⊈ localization(SID, SN, RID, RN)
- sales(SID, PID, …,P,…), P>80, P<150 ⊈ sales(SID, PID, …,P,…), P>100, P<150
- sales(SID, PID, …,P,…), product(PID,PN,…)
  ⊆ sales(SID, PID, …,P,…) when projecting to sales attributes

Example

$q_1 \subseteq q_2$?

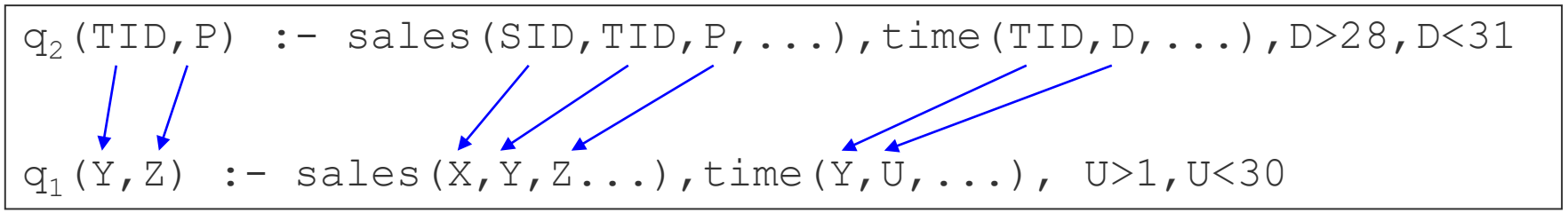


Mapping: A→A, B→B, C→C, D→D



Mapping: A→B, B→C, C→D, D→E

Example



CM: $SID \rightarrow X,\ TID \rightarrow Y,\ P \rightarrow Z,\ D \rightarrow U$
$h(D)=U$

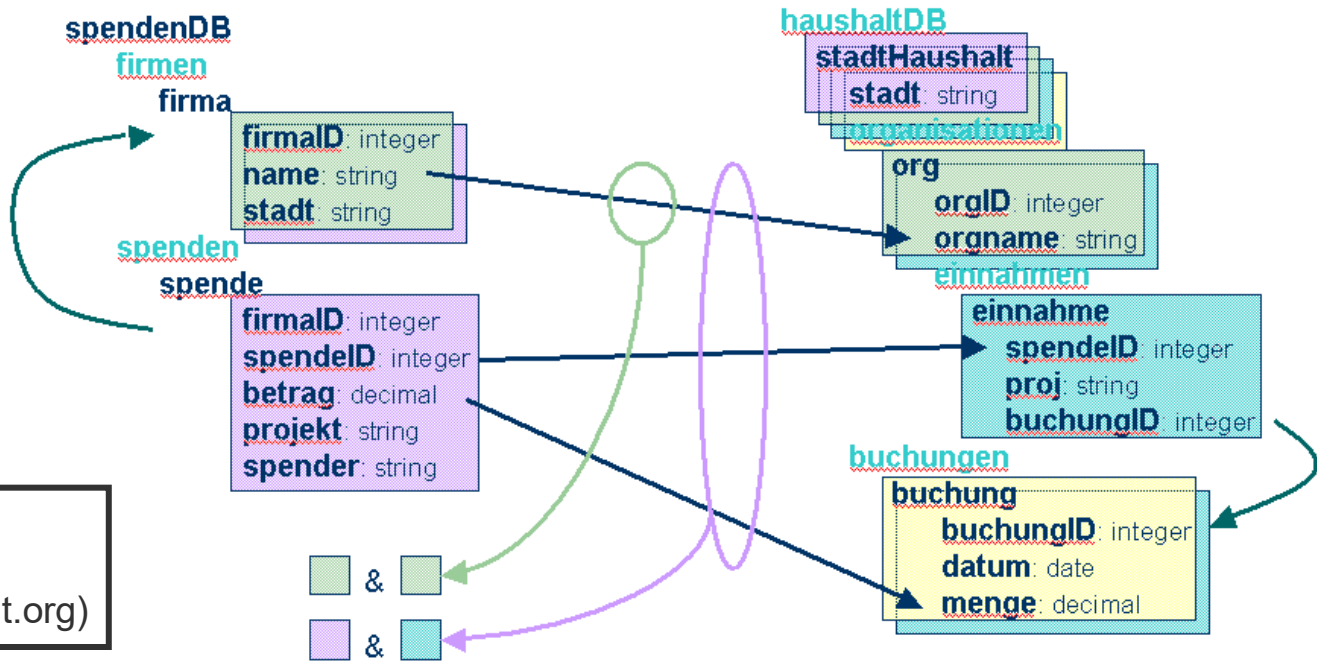
Aber: $U>1 \wedge U<30 \not\rightarrow h(D)>28 \wedge h(D)<31$

$$q_1 \not\subseteq q_2$$

Felix Naumann
Data Integration
Summer 2025

Reprise: Query Generation from Schema Mapping

Beobachtung:
Interpretation als
containment



$\pi_{name}(\text{spendenDB.firmen}) \subseteq \pi_{orgname}(\text{haushaltDB.stadtHaushalt.org})$

$\pi_{name,spendeID,betrag}(\text{spendenDB.firmen} \bowtie \text{spendenDB.spenden}) \subseteq \pi_{orgname,spendeID,menge}(\text{haushaltDB.stadtHaushalt.org.einnahmen} \bowtie \text{haushaltDB.stadtHaushalt.buchungen})$

Query Rewriting

- Rewriting from global query to set of local queries

- In general:
  - Check every combination of views for containment
  - An infinite number of combinations, since a view can also be included in a rewriting several times.

- Improvements:
  - Theorem: Rewrite with a maximum number of views as there are relations in query (without range predicates).
  - Clever pre-selection of views: usability
    - Bucket algorithm

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. **Local as View (LaV)**
   - Modeling
   - Applications
   - Query Processing
   - Containment
   - **Bucket Algorithm**
6. Global Local as View (GLaV)
7. Comparison

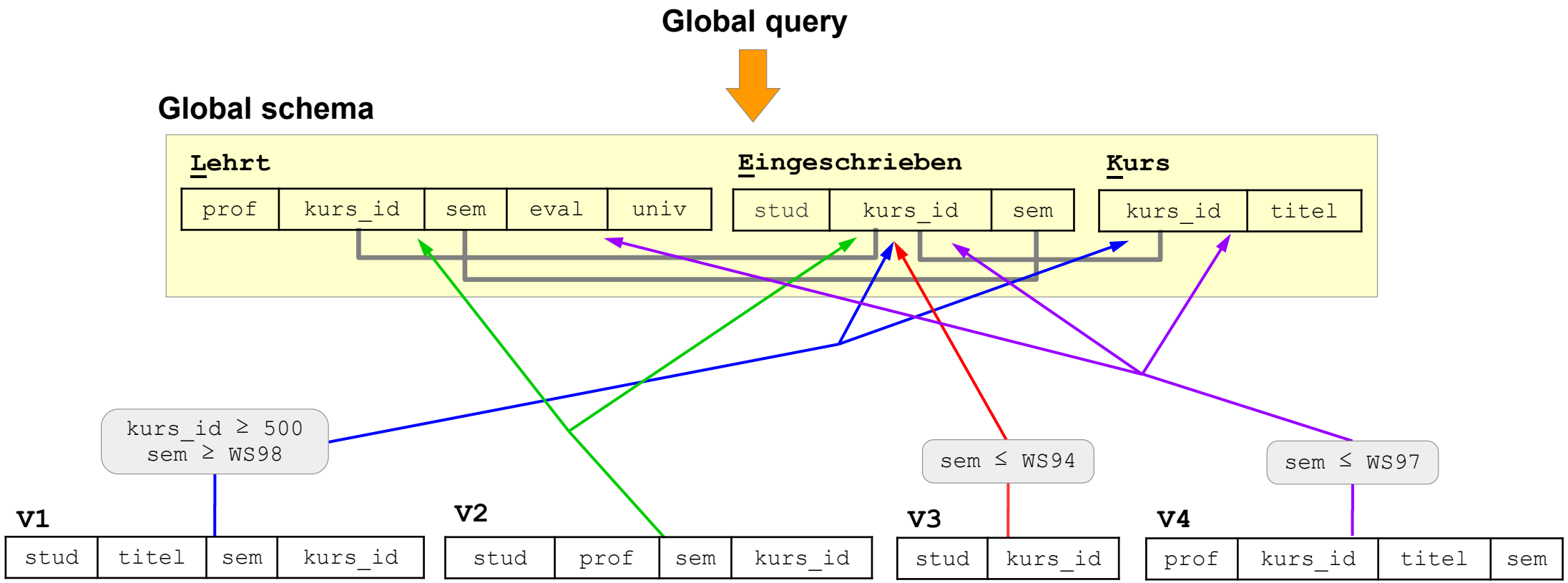
Felix Naumann
Data Integration
Summer 2025

LaV – Bucket Algorithm (BA)

- Problem:
  - Check exponentially many combinations for containment.
  - Even the check itself is an NP-complete problem.
    - however, there are fast algorithms
    - usually the combinations are not large (number of relations)

- Idea for improvement
  - Reduction of the number of combinations through pre-selection
  - Each relation of the query is given a bucket.
  - Step 1: Add to each bucket all views that can be used for the relation.
  - Step 2: Check all query rewrite combinations that contain exactly one view from each bucket.

LaV – BA – Example



V1 (stud, titel, sem, kurs_id) :- E(stud,kurs_id,sem), K(kurs_id,titel), kurs_id ≥ 500, sem ≥ WS98

V2 (stud, prof, sem, kurs_id) :- E(stud, kurs_id, sem), L(prof, kurs_id, sem)

V3 (stud, kurs_id) :- E(stud, kurs_id, sem), sem ≤ WS94

V4 (prof, kurs_id, titel, sem) :- L(prof, kurs_id, sem), K(kurs_id, titel), E(stud, kurs_id, sem), sem ≤ WS97

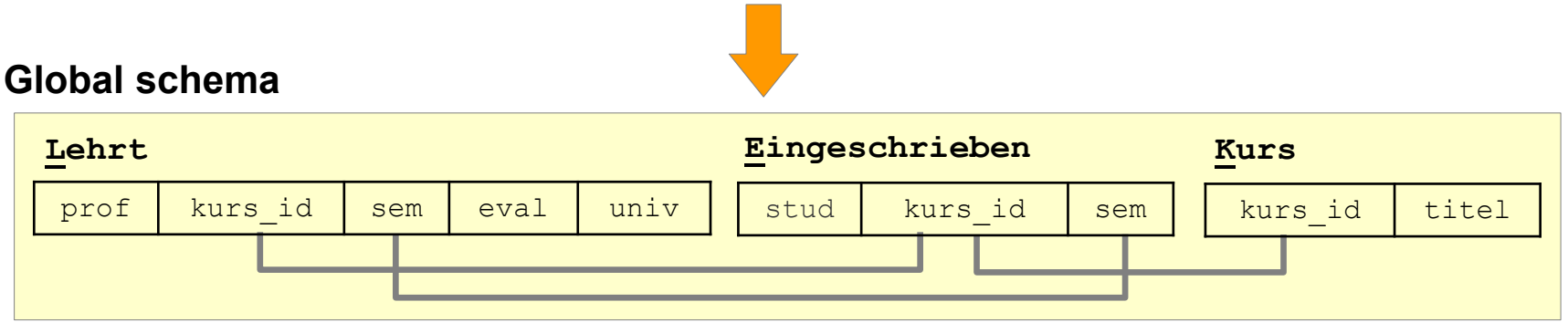
LaV – BA – Example

Globale query Q:

```
SELECT E.stud, K.kurs_id, L.prof
FROM   Lehrt L, Eingeschrieben E, Kurs K
WHERE  E.kurs_id = L.kurs_id
AND    E.sem = L.sem
AND    K.kurs_id = E.kurs_id
AND    E.sem ≥ WS95 AND kurs_id ≥ 300
```

```
Q(stud, kurs_id, prof) :- L(prof, kurs_id, sem), E(stud,kurs_id,sem),
                          K(kurs_id, titel), sem ≥ WS95, kurs_id ≥ 300
```

Global schema



Felix Naumann
Data Integration
Summer 2025

LaV – BA – Example

V1 (stud, titel, sem, kurs_id) :- E(stud,kurs_id,sem), K(kurs_id,titel), kurs_id ≥ 500, sem ≥ WS98

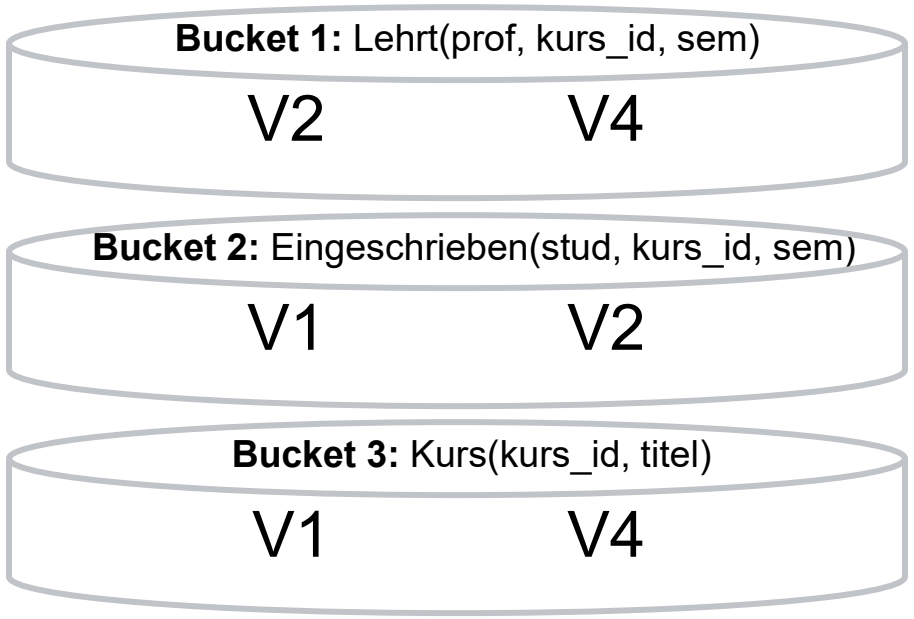
V2 (stud, prof, sem, kurs_id) :- E(stud, kurs_id, sem), L(prof, kurs_id, sem)

V3 (stud, kurs_id) :- E(stud, kurs_id, sem), sem ≤ WS94

V4 (prof, kurs_id, titel, sem) :- L(prof, kurs_id, sem), K(kurs_id, titel), E(stud, kurs_id, sem), sem ≤ WS97

Query:

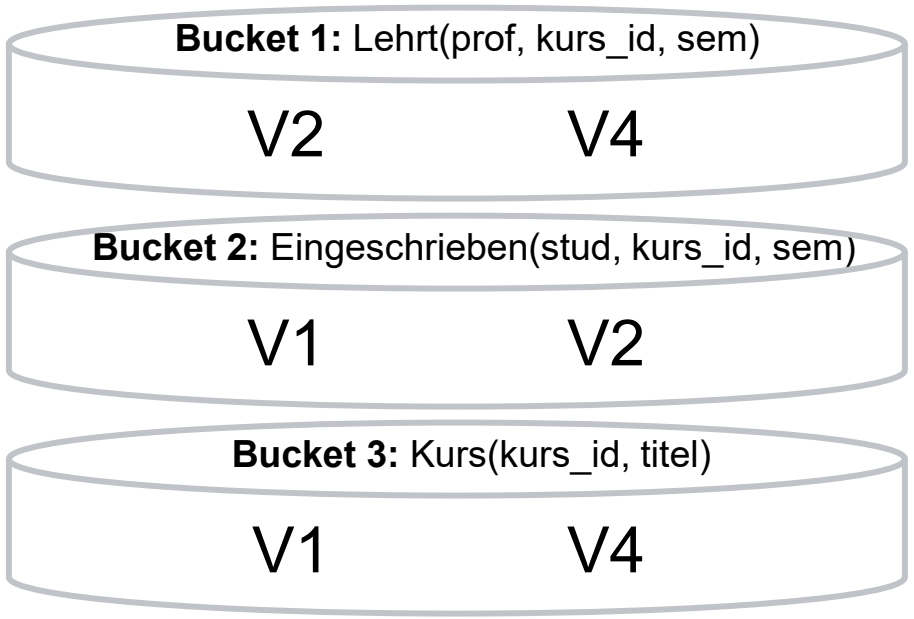
Q (stud, kurs_id, prof) :-
L(prof, kurs_id, sem),
E(stud,kurs_id,sem),
K(kurs_id, titel),
sem ≥ WS95,
kurs_id ≥ 300



LaV – BA – Example

V1 (stud, titel, sem, kurs_id) :- E(stud,kurs_id,sem), K(kurs_id,titel), kurs_id ≥ 500, sem ≥ WS98
V2 (stud, prof, sem, kurs_id) :- E(stud, kurs_id, sem), L(prof, kurs_id, sem)
V3 (stud, kurs_id) :- E(stud, kurs_id, sem), sem ≤ WS94
V4 (prof, kurs_id, titel, sem) :- L(prof, kurs_id, sem), K(kurs_id, titel), E(stud, kurs_id, sem), sem ≤ WS97

Query: Q (stud, kurs_id, prof) :- L(prof, kurs_id, sem), E(stud,kurs_id,sem), K(kurs_id, titel),
sem ≥ WS95, kurs_id ≥ 300



Combinations

V2, V1, V1 → V1,V2
V4, V1, V4 → Ø
V2, V2, V4 → V2,V4
V2, V2, V1 → V1,V2
V2, V1, V4 → Ø
V4, V1, V1 → Ø
V4, V2, V4 → V2,V4
V4, V2, V1 → Ø

LaV – BA – Example

V1 (stud, titel, sem, kurs_id) :- E(stud,kurs_id,sem), K(kurs_id,titel), kurs_id ≥ 500, sem ≥ WS98
V2 (stud, prof, sem, kurs_id) :- E(stud, kurs_id, sem), L(prof, kurs_id, sem)
V3 (stud, kurs_id) :- E(stud, kurs_id, sem), sem ≤ WS94
V4 (prof, kurs_id, titel, sem) :- L(prof, kurs_id, sem), K(kurs_id, titel), E(stud, kurs_id, sem), sem ≤ WS97

Query: Q (stud, kurs_id, prof) :- L(prof, kurs_id, sem), E(stud,kurs_id,sem), K(kurs_id, titel),
sem ≥ WS95, kurs_id ≥ 300

Combinations

V2, V1, V1 → V1,V2
V4, V1, V4 → Ø
V2, V2, V4 → V2,V4
V2, V2, V1 → V1,V2
V2, V1, V4 → Ø
V4, V1, V1 → Ø
V4, V2, V4 → V2,V4
V4, V2, V1 → Ø

Result of algorithm:
V1,V2 ∪ V2,V4

LaV – BA – Example

Source 1:

```
CREATE VIEW V1 AS
SELECT E.stud, K.titel, K.sem, K.kurs_id
FROM   Eingeschrieben E, Kurs K
WHERE E.kurs_id = K.kurs_id
AND      K.kurs_id LIKE „%VL_“
AND      E.sem ≥ WS98
```

Source 2:

```
CREATE VIEW V2 AS
SELECT E.stud, L.prof, E.sem, L.kurs_id
FROM   Eingeschrieben E, Lehrt L
WHERE E.kurs_id = L.kurs_id
AND      E.sem = L.sem
```

Source 4:

```
CREATE VIEW V4 AS
SELECT L.prof, K.kurs_id, K.titel, E.sem
FROM   Eingeschrieben E, Kurs K, Lehrt L
WHERE E.kurs_id = K.kurs_id
AND      E.kurs_id = L.kurs_id
AND      L.sem = E.sem
AND      E.sem ≤ WS97
```

Query Q:

```
SELECT E.stud, K.kurs_id, L.prof
FROM   Lehrt L, Eingeschrieben E, Kurs K
WHERE E.kurs_id = L.kurs_id
AND      E.sem = L.sem
AND      K.kurs_id = E.kurs_id
AND      E.sem ≥ WS95
```

Rewritten query Q‘

V1,V2 ∪ V2,V4**:**

```
SELECT V1.stud, V1.kurs_id, V2.prof
FROM   V1, V2
WHERE V1.sem = V2.sem
AND      V1.kurs_id = V2.kurs_id

UNION

SELECT V2.stud, V2.kurs_id, V2.prof
FROM   V2, V4
WHERE V2.sem = V4.sem
AND      V2.kurs_id = V4.kurs_id
AND      V2.prof = V4.prof
AND      V2.sem ≥ WS95
```

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
6. **Global Local as View (GLaV)**
7. Comparison

Felix Naumann
Data Integration
Summer 2025

Global-Local-as-View (GLAV)

- Combination of GaV and LaV
  - GaV:
    - Global relation = view on local relations
    - (Global relation $\supseteq$ view on local relations)
  - LaV:
    - View on global relations = local relation
    - View on global relations $\supseteq$ local relation
  - GLaV:
    - View on global relations = view on local relations
    - View on global relations $\supseteq$ view on local Relations
- Also „BaV“: Both-as-View

Query Planning

- GaV:
  - View unfolding
  - Standard techniques from relational databases
- LaV
  - Query containment and answering queries using views
  - Multiple algorithms, also serves other applications
  - Difficult planning
- GLaV
  - First, query planning with the views on the global schema
    - The left sides of the correspondences
  - Then, unfolding with the views on the local schemata
    - The right sides of the correspondences
- Then distributed optimization

Overview

1. Motivation
2. Correspondences
3. Overview: Query Planning
4. Global as View (GaV)
   - Modeling
   - Query Processing
5. Local as View (LaV)
   - Modeling
   - Applications
   - Query Processing
   - Containment
6. Global Local as View (GLaV)
7. **Comparison**

Felix Naumann
Data Integration
Summer 2025

Comparison GaV / LaV

Modeling

- GaV
  - Each global relation is defined as a view on one or more relations from one or more sources.
  - Mostly UNION via multiple sources
  - Constraints on local sources cannot be modeled.
- LaV
  - Global relations are defined by several views.
  - Definition often only in combination with other global relations
  - Constraints on global relations cannot be modeled.
- GLaV
  - Global and local constraints are possible
  - Concepts can be constrained locally or globally

Comparison GaV / LaV

- Query processing
  - GaV:
    - Query rewriting: Simple view unfolding
    - One large, rewritten query
    - Interesting optimization problems
  - LaV
    - Query rewriting: *Answering queries using views*
    - UNION on many possible rewritten queries
    - Multiple algorithms
    - Multiple applications
    - Even more interesting optimization problems
- Flexibility
  - GaV: Views relate relations among several sources
  - LaV: Each view refers to only one source

Literature

Good summary for LaV and further reading:

- [Levy01] Alon Y. Halevy: Answering queries using views: A survey, in VLDB Journal 10: 270-294, 2001.

Rather theoretical

- [Ull00] Jeffrey D. Ullman: Information Integration Using Logical Views. TCS 2000: 189-210
- [Hull97] Managing Semantic Heterogeneity in Databases: A Theoretical Perspective. Richard Hull. PODS 1997 tutorial
- [Ull97] Jeffrey D. Ullman: Information Integration Using Logical Views. ICDT 1997: 19-40
- [CM77] Ashok K. Chandra and Philip M. Merlin. Optimal implementation of conjunctive queries in relational data bases. In Conference Record of the Ninth Annual ACM Symposium on Theory of Computing, pages 77-90, Boulder, Colorado, 2-4 May 1977.
- [LMSS95] Alon Y. Levy, Alberto O. Mendelzon, Yehoshua Sagiv, Divesh Srivastava: Answering Queries Using Views. PODS 1995: 95-104